

Funkcionální a logické programování



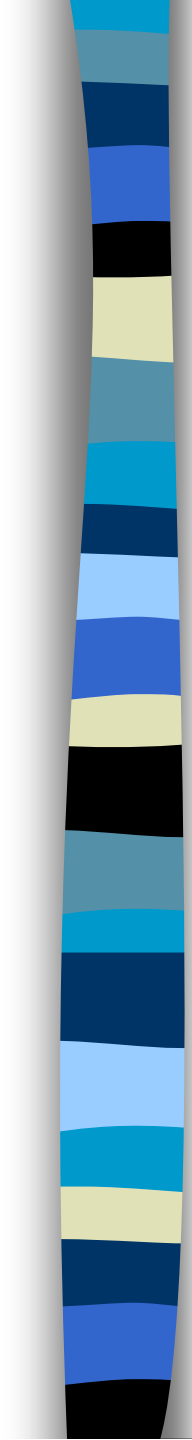
Dušan Kolář

kolar@fit.vutbr.cz

+420-54-114 1238

Bodový zisk

□ Půlsemestrální zkouška	20
Projekt 1 (FP)	12 / 4
Projekt 2 (LP)	8 / 3
<i>Zápočet</i>	<i>20 / 7 nestačí!</i>
Závěrečná zkouška	60 (25)
<i>kompenzace 20/60 za půlsem. zk.</i>	
□ CELKEM	100 (32)



Důležitá data

- Odevzdání projektů (*další info IS a cvičení*)
 - projekt 1: viz IS, Moodle, ...
 - projekt 2: viz IS, Moodle, ...
- Půlsestrální zkouška *7. týden*
 - *23. 3. 2025*
- Závěrečná zkouška (semestr 9. 5.)
 - termín 1: *bude určen globálně*
 - termín 2: *bude určen globálně*
 - termín 3: *bude určen globálně*

Rozvrh – WIS

□ **Přednášky:**

pondělí

– učebna

E112

– čas

13:00-15:50

– 6x3 = 18, půls. zk. 2, 1x3, (před)poslední týden 1x3

□ **Počítače:**

úterý (jiné dny možné)

– učebna

N204-N205

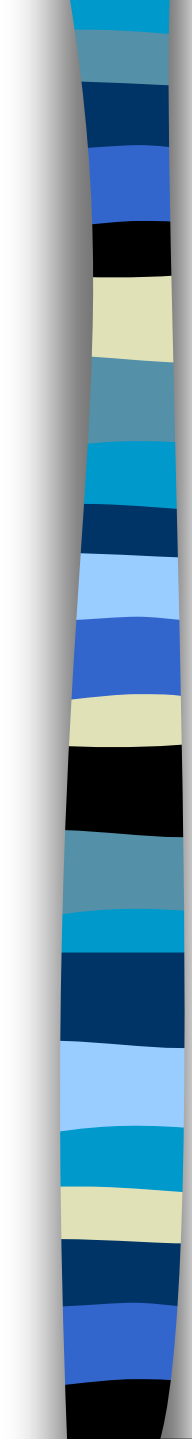
– čas

16:00-19:50 (út)

Přihlášení! Výuka již běží!

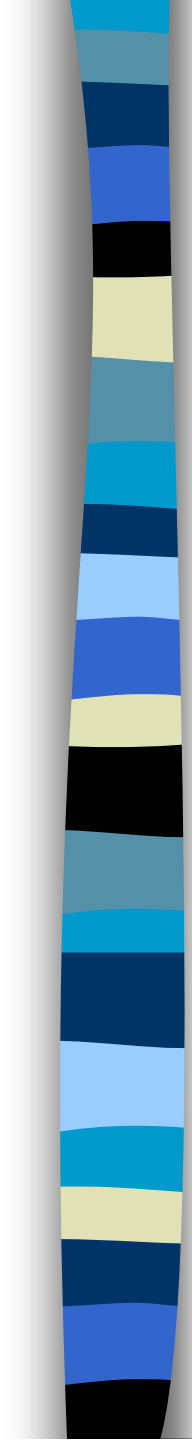
Záznamy přednášek

- Budeme ukládat
(pokud to pojede – zvuk i obraz)
 - Zveřejnění po zpracování (pozor i 3 týdny!)
 - **Nemá význam, jaký se tomu přikládá!**
 - **Nový styl myšlení při tvorbě programu je třeba si odzkoušet!**
- ***Nejedná se o encyklopedický předmět – nejde ani o algoritmizaci, ale o konstrukci, která je neimperativní...!!!***



Doporučení

- Sledujte [www stránky](#)
 - Všechny údaje nejprve tam
 - Sám od tam opisují data
- Chodte na přednášky, **cvičení – body apod.**
 - v průběhu si zkoušejte, co se říká
- Programujte, programujte, programujte
- ***Nejde o algoritmizaci, ale o zápis!***
 - Algoritmy znáte/jsou triviální
- Projekt $\neq$ jedno řešení všech



Individuální projekt

□ Není to

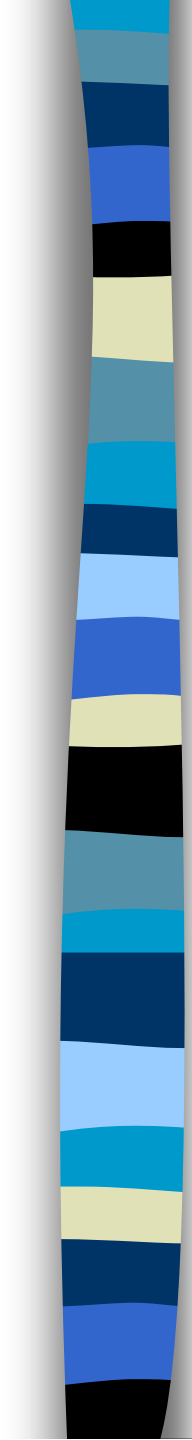
- že odjedu na Saharu a tam to spáchám
- že to stáhnu z netu a upravím
- že se o práci podělím s kolegou/y

□ Může být to

- že s kolegy provedu diskusi na dané téma, ale naimplementuje každý **sám**
- podívám se na net, jak to kdo řešil, ale své řešení vytvořím **sám**

Individuální projekt

- Je výrazně jednodušší, než týmový
 - žádný „parsec“
- Odbourává závislost na kolezích
 - špatné zkušenosti, aspoň 2 týmy ročně
- Týmové jsou v jiných předmětech
- Umožňuje lepší organizaci
- **Přinutí si to všechny osahat** 😊



Hodnocení individuálního projektu

- Bere v potaz rozdílnost složitosti jednotlivých zadání
- Je mírnější, než u týmových projektů
- Umožňuje získ bonusových bodů za zvolení přístupu k řešení problému, který pozitivně vybočuje z řady

*Uznání z loňska **není letos možné!***

Důvody pro studium FJ (1)

□ S programy se lépe matematicky pracuje

– referenční transparence

- v daném kontextu představuje proměnná vždy tutéž hodnotu

»

$x := x + 1$

$\{ x = 3 \}$

$\{ x = 4 \}$

imperativní styl

proměnná $\equiv$
adresa hodnoty

– stejné lze vyjádřit stejným (optimalizace)

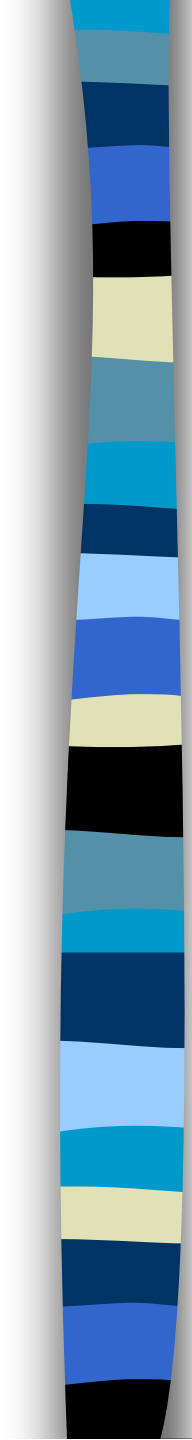
» $x - x = f(1) - f(1)$

Důvody pro studium FJ (2)

- FJ mají lepší mechanismus abstrakce
 - díky možnosti abstrakce jsou programy ve FJ stručnější
(problém: kryptografické programy)
 - funkce vyššího řádu (funkce jako hodnota, ne ukazatel na funkci)
 - » $\Sigma = 0 + n_1 + n_2 + \dots$
 - $\Pi = 1 * n_1 * n_2 * \dots$
 - $\heartsuit = C \heartsuit n_1 \heartsuit n_2 \heartsuit \dots$
 - » `compute(C,  $\heartsuit$ ,  $n_1$ ,  $n_2$ , ...)`

Důvody pro studium FJ (3)

- FP umožňuje nové algoritmické přístupy
 - lazy evaluation X eager/strict evaluation
 - » možnost práce s potenciálně nekonečnými datovými strukturami
 - » možnost oddělení dat od řízení (nemusíme se starat o to, jak proběhne vyhodnocení)
 - » sdílení mezivýsledků/memoization

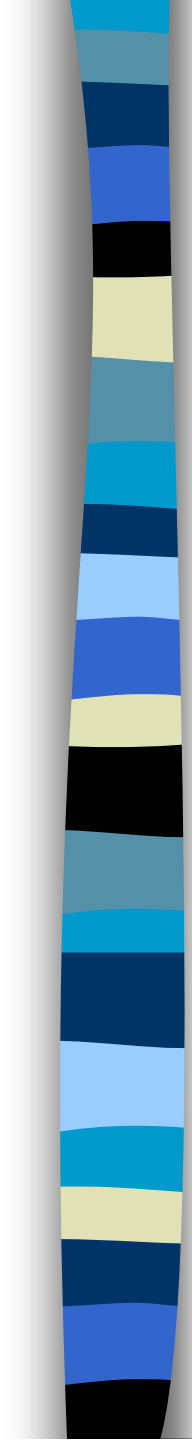


Důvody pro studium FJ (4)

- FP umožňuje nové přístupy k vývoji programů
 - možnost dokazování programů
 - možnost transformace programů na základě algebraických vlastností

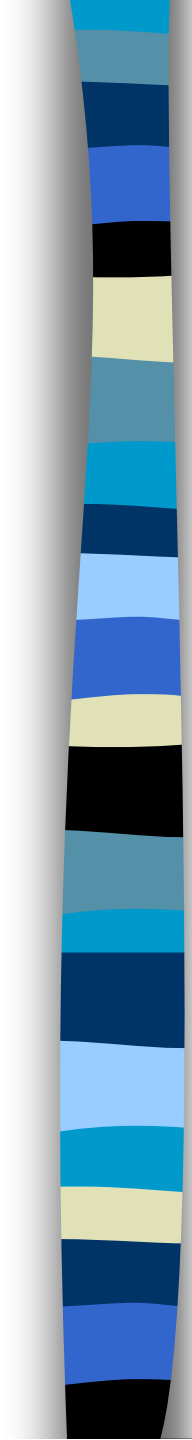
Důvody pro studium FJ (5)

- FP umožňují využít (masivně) paralelního výpočtu
 - „jednoduchá“ dekompozice programu na části, které lze vyhodnocovat paralelně
 - problém: paralelismu je potenciálně příliš mnoho



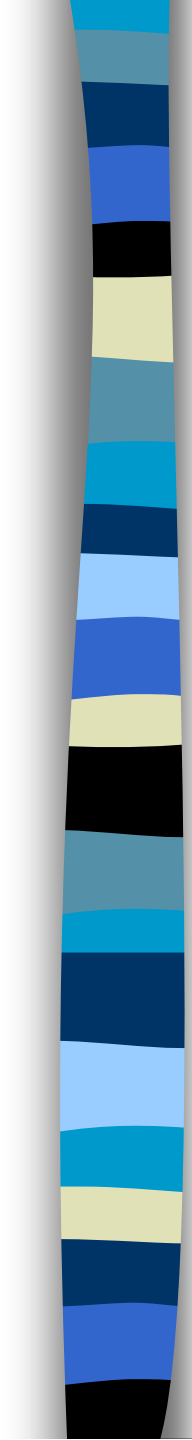
Důvody pro studium FJ (6)

- FP je důležité pro některé oblasti aplikace informatiky
 - umělá inteligence
 - specifikace (jazyk Z), modelování, rychlé prototypování
- Možnost využít i pro „tradiční“ oblasti informatiky



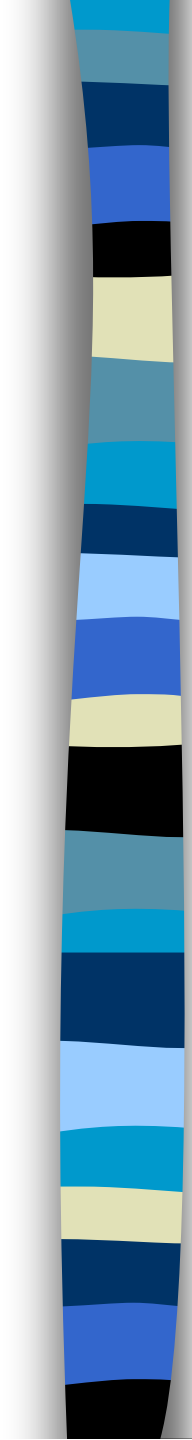
Důvody pro studium FJ (7)

- FP má blízký vztah k teoretické informatice
 - sémantika programovacích jazyků
 - teorie složitosti algoritmů
 - teorie vyčíslitelnosti



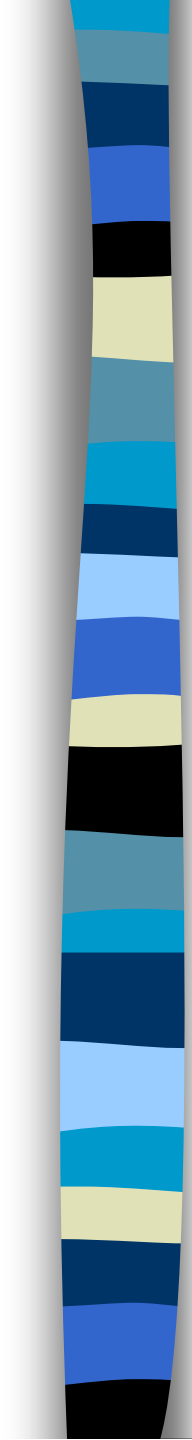
Důvody pro studium FJ (8)

- FP je zajímavou a myšlení rozvíjející činností pro studenty informatiky!
 - Jiný pohled na programování
 - Odstranění zažitých dogmat z procedurálních jazyků
 - Java
 - C++
 - Scala



Literatura - FP

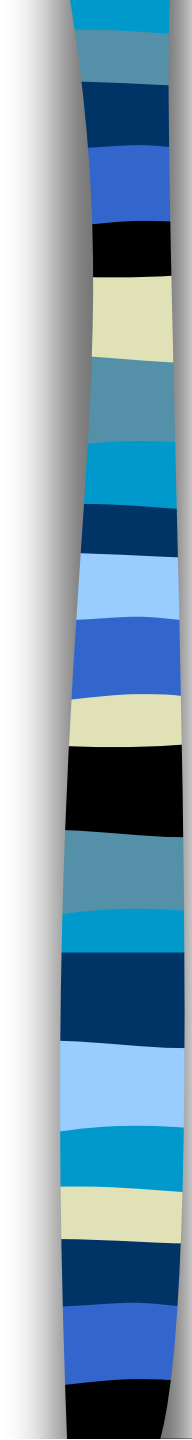
- Bird R., Wadler P.: *Introduction to Functional Programming*, Prentice Hall, 1988, ISBN 0-13-484189-1
- Thompson S.: *Haskell, The Craft of Functional Programming*, Addison Wesley, 1999, ISBN 0-201-34275-8
- **WWW stránky:**
 - <http://www.haskell.org>



Literatura - LP

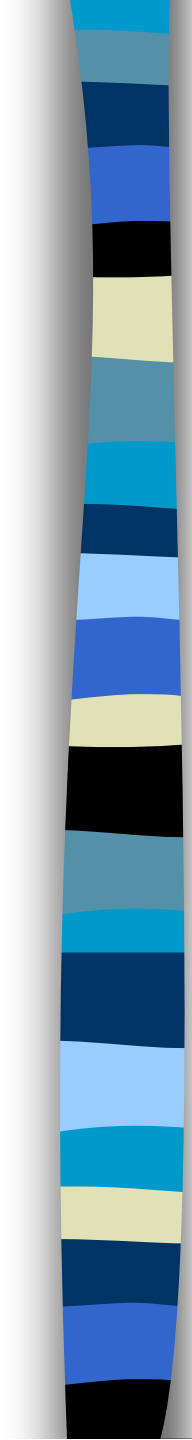
- Hill P., Lloyd J.: *The Gödel Programming Language*, MIT Press, 1994, ISBN 0-262-08229-2
- Bieliková M., Návrát P.: *Funkcionálne a logické programovanie*, 2000, FEI STU Bratislava

- **WWW stránky:**
 - <http://www.swi-prolog.org>
 - <http://www.csupomona.edu/~jrfisher/www/>
 - http://cs.wwc.edu/~cs_dept/KU/PR/Prolog.html



Přehled přednášek

1. Úvod do FP
2. Haskell - úvod, seznamy
3. Uživatelské datové typy, typové třídy, pole
4. Vstupy/výstupy
5. Praktické úkoly
6. Dokazování ve FP
7. Denotační sémantika
8. Prolog – úvod
9. Seznamy, operátor řezu, řazení
10. Praktiky v Prologu 1
11. Praktiky v Prologu 2
12. Praktiky v Prologu 3
13. Gödel
14. CLP



Úvod do funkcionálního programování

Cíle oddílu



□ Úvod do FP

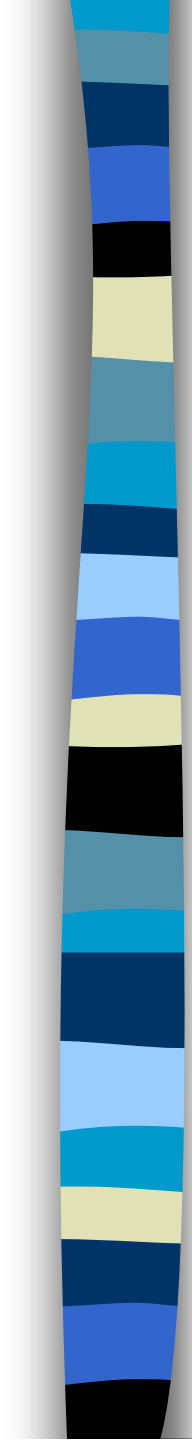
- Imperativní a deklarativní programování

□ Důvody studia FP

□ Lambda kalkul – rekapitulace

- Pro začátečníky je vhodné nastudovat např. Češka, Motyčková, Hruška: Vyčíslitelnost a složitost, či jiné publikace o lambda kalkulu

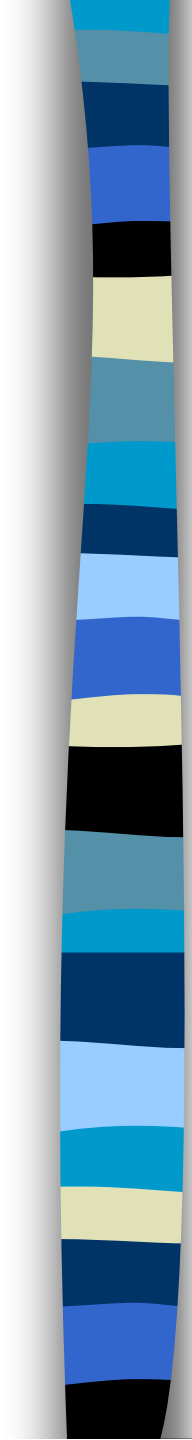
□ Důležité pojmy z algebry



Klasifikace programovacích jazyků

□ Imperativní jazyky

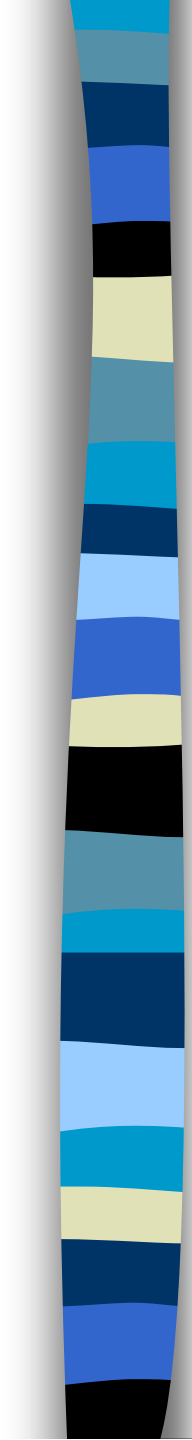
- program má implicitní **stav**, který se modifikuje
- explicitní pojem **pořadí** příkazů
- vyjadřujeme to, **jak** se má program vyhodnocovat
- vhodné pro tradiční architektury (von N.)
- většina současných jazyků



Klasifikace programovacích jazyků (pokr.)

□ Deklarativní jazyky

- program **nemá** implicitní stav, stavové informace se udržují explicitně
- program je tvořen **výrazy** (termy) - nikoliv příkazy
- opakované provádění se vyjadřuje **rekurzí**
- vyjadřujeme **co** se má spočítat



Funkcionální (aplikativní) jazyky

□ Základní model:

- matematický pojem funkce aplikované na argumenty a vypočítávající jediný výsledek
⇒ výsledek je deterministický

□ Čistě funkcionální jazyky:

- FP, **Haskell**, Miranda, Hope, Orwell

□ Hybridní jazyky (imperativní):

- Lisp, Scheme, SML (a jeho deriváty)

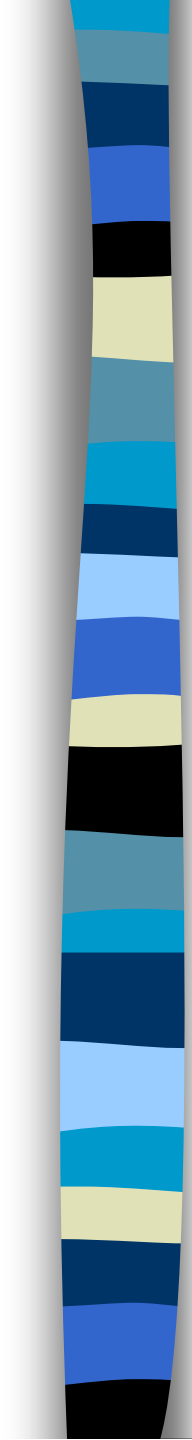
Relační (logické) jazyky

□ Základní model:

- **relace** (predikát)
- výpočet probíhá „nedeterministicky“ (s navracením)

□ Jazyky:

- **Prolog**, Parlog, **Goedel**
- CLP



Historie funkcionálních jazyků

1930 Alonzo Church

čistý λ -kalkul
typovaný λ -kalkul

1958 Mac Carthy

LISP

1965 P. Landin

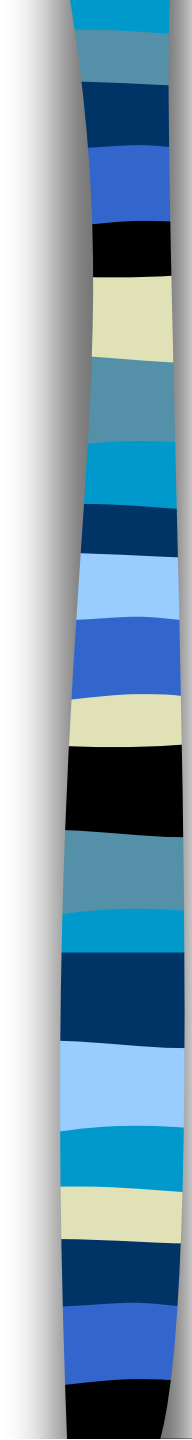
ISWIM
předchůdce ML

1978 R. Milner

ML - metajazyk pro
systém podporující
dokazování
původní typový
systém

1985 D. Turner

Miranda
lazy evaluation



Historie funkcionálních jazyků (pokr.)

1987

Haskell - modulární,
lazy evaluation,

1989 A. Appel
& D. Mac Queen
Standard ML

1990 X. Leroy
Caml Light

1998

Haskell 98
standardizovaná
verze jazyka
(sloučení 4 hlavních
implementací)

2003/4

Haskell - knihovny

λ -kalkul

□ Matematický model FJ

– FJ lze na něj převést, bez výhrady

□ Syntax:

– výraz	→	proměnná
		(výraz výraz)
		(λ proměnná.výraz)

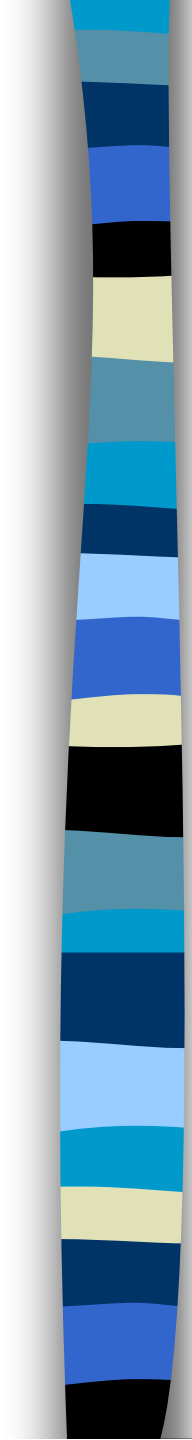
Proměnná

- Označuje jakoukoliv hodnotu
 - typovaný X beztypový přístup
- **Vázaná a volná proměnná**
 - $\lambda f. (f\ x)$
 - x je v $(f\ x)$ volná
 - f je v $(f\ x)$ vázaná

Aplikace

□ $(e_1 \ e_2)$

- představuje aplikaci funkce e_1 na výraz e_2
- výsledek přitom může být dále aplikovatelný



Abstrakce

□ $(\lambda x.e)$

- představuje funkci s parametrem x a tělem e

Substituce

□ $e_1[e_2/x]$

- substituce výrazu e_2 za všechny volné výskyty proměnné x ve výrazu e_1

Platná substituce

- $e_1[e_2/x]$
 - libovolná **volná** proměnná v e_2 se nesmí stát vázanou
- $(\lambda f.f\ x)[y/x] = (\lambda f.f\ y)$
- $(\lambda f.f\ x)[(f\ y)/x]$ **je neplatná substituce**

α - konverze

- Přejmenování proměnných
 - $(\lambda x.e) \leftrightarrow (\lambda x'.e)[x'/x]$
 - substituce musí být platná!
- Využití
 - unifikace (porovnání)
 - zlepšení čitelnosti výrazů

β - konverze

- Vyhodnocení abstrakce
 - $(\lambda x. e_1) e_2 \leftrightarrow e_1[e_2/x]$
 - substituce musí být platná
- Operační charakter
 - mění formu výrazu, ale ne jeho hodnotu
 - ta je dána prvotním zápisem
 - globální strategie vyhodnocení (postupu)

η - konverze

□ Odstranění abstrakce

- $(\lambda x. f \ x) \leftrightarrow f$
- x nesmí být volná v f
 - Funkce jsou ekvivalentní pokud pro všechny hodnoty dávají stejný výsledek
- Umožňuje definovat **řezy** funkcí
 - $(+1) = (\lambda x. x + 1)$

Normalizační teorémy

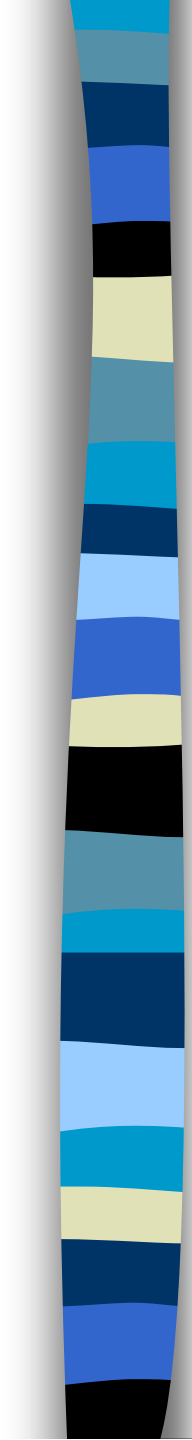
- Church-Rosserovy teorémy
- **redex** - **reducible expression**: výraz, který lze dále redukovat
 - α/β - redex
- **normální forma** - výraz je v normální formě pokud neobsahuje žádný β - redex

Teorém I

- Pokud $e_1 \Leftrightarrow e_2$, pak existuje výraz e takový, že
 - $e_1 \rightarrow e$
 - $e_2 \rightarrow e$

Teorém II

- Pokud $e_1 \rightarrow e_2$ a e_2 je v normální formě, pak existuje taková redukční posloupnost, že z e_1 je možné se „bezpečně“ dostat do e_2
(normální redukční posloupnost)



Důsledek

- Pokud existuje normální forma, lze k ní dojít normální redukční posloupností (leftmost outermost redex)
 - Určuje způsoby vyhodnocení ve FP
 - Nevyhodnocuje se pod λ -abstrakcemi
 - využívá se zejména v programování u striktních jazyků

Funkce - matematický pojem

□ Zobrazení množiny A do množiny B

– $f: A \rightarrow B$

- A - definiční obor
- B - obor hodnot
- $f(a) = b$

– $f(x_1) = y_1 \ \&\& \ f(x_1) = y_2 \Rightarrow y_1 = y_2$

Vybrané typy funkcí

□ Injektivní

- různé prvky A se zobrazují na různé prvky B
 $f(a) = f(a') \Leftrightarrow a = a'$

□ Surjektivní

- všechny prvky oboru hodnot mají svůj vzor
 $\forall b \in B \exists a \in A: f(a) = b$

□ *Bijektivní*

- injektivní + surjektivní

Kompozice a inverze funkcí

□ Kompozice

- $f: A \rightarrow B, g: B \rightarrow C$
- $(g \circ f)(a) = g(f(a))$

□ Inverze

- $f: A \rightarrow B, f^{-1}: B \rightarrow A$
- $f^{-1}(f(a)) = a$
 - existuje ke každé injektivní funkci
 - každá bijektivní funkce je invertibilní

Binární operace na A

□ $\oplus: (A \times A) \rightarrow A$

- asociativita
- komutativita
- levá/pravá identita

» $e \oplus x = x$

- levá/pravá nula

» $z \oplus x = z$

- levý/pravý inverzní prvek

» $x \oplus y = e$

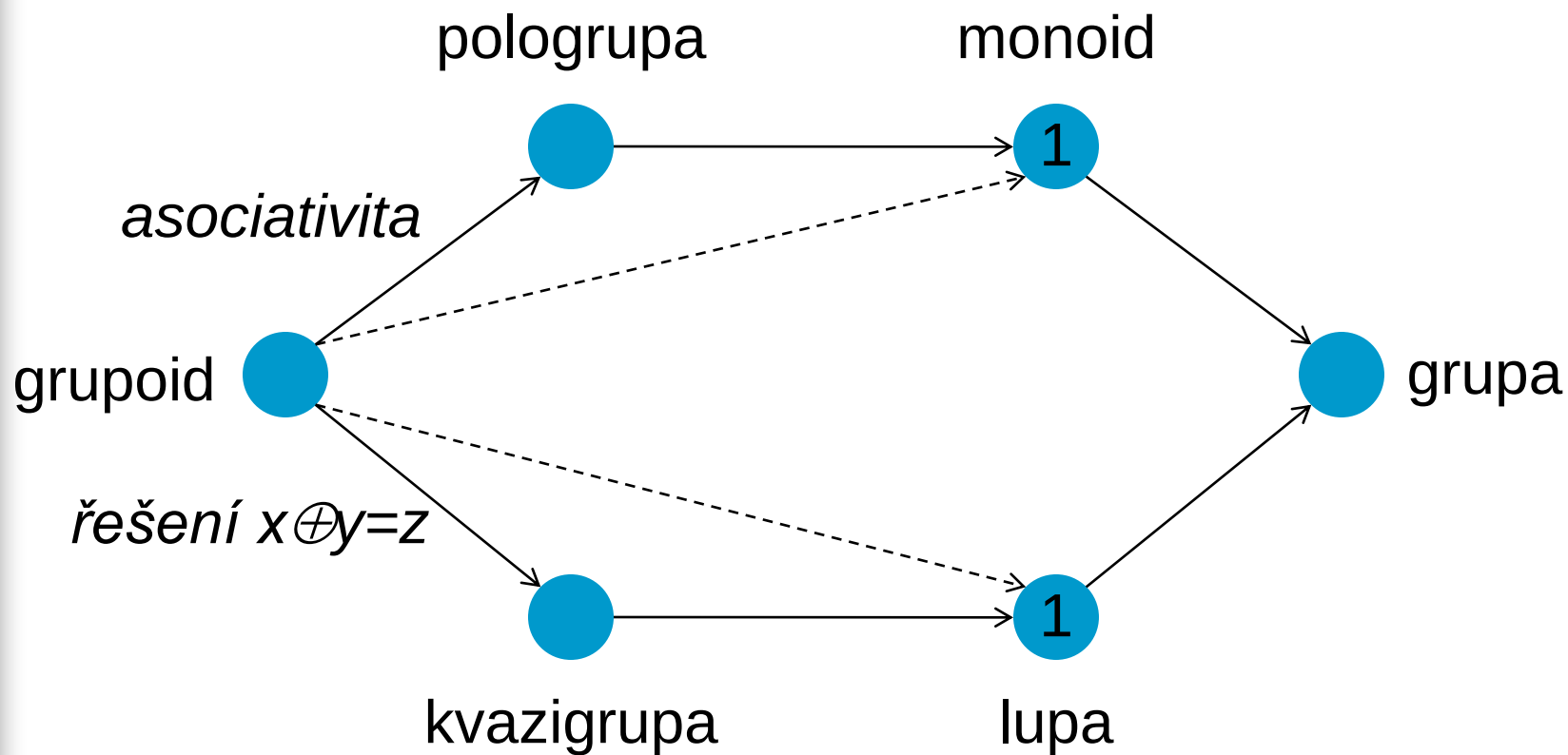
$a \oplus a'$
 $(A, \oplus)$ pologrupa

monoid

grupa

Abelovská grupa

Od grupoidu ke grupě



Důležité pojmy

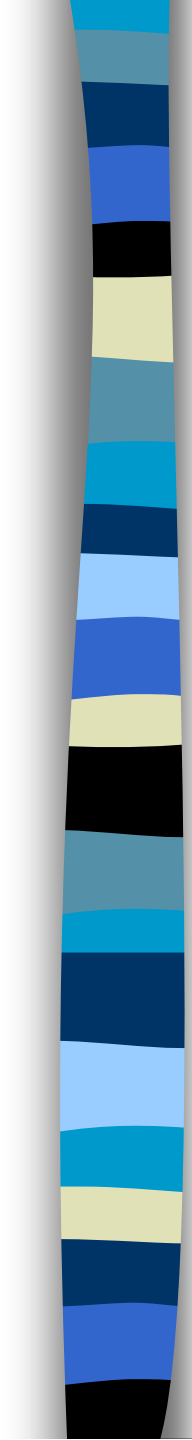
- Imperativní, deklarativní
- Lambda kalkul
 - Definice
 - Konverze/redukce
 - Substituce
- Kompozice, inverze, asociativita, komutativita
- Pologrupa, monoid, grupa



Úkoly



- Navrhnete reprezentaci čísel v lambda kalkulu
- Navrhnete běžné operace nad těmito čísly
- Ukažte, že platí obecně známé rovnosti:
 - $x+y = y+x$
 - $x+0 = x$
 - ...



Haskell - úvod, seznamy

Cíle oddílu



- Přehled základních datových typů jazyka Haskell
- Deklarace funkcí v jazyce Haskell
- Rekurze
- Práce se seznamy

Co a kde ...

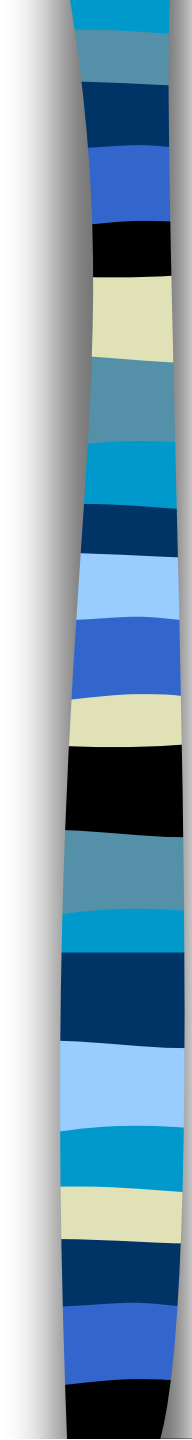
□ Haskell

- překladač (kompilátor) – **ghc**, **nhgc**, **jhc**
- interaktivní kompilátor – **ghci**
- <http://www.haskell.org>

□ *Hugs*

- *interpret virtuálního kódu*
- <http://www.haskell.org>

□ *Linux, Q:\FLP\...*



Základní datové typy

□ Int

- 45, 890

□ Char

- 'a', 'X'

□ Bool

- True, False

□ Float

- 3.14159, 234.2

Seznamy

□ Konstruktor

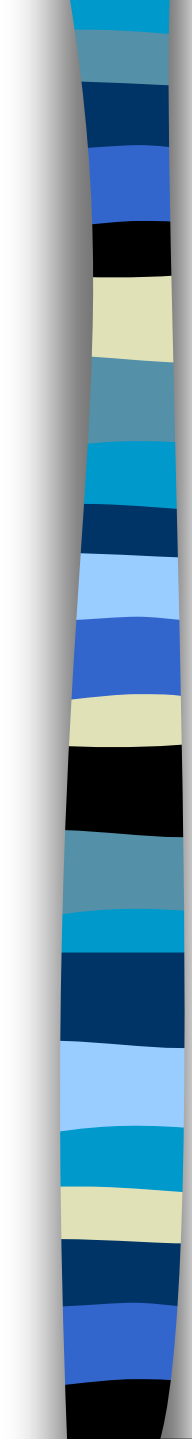
- $:$
- $[]$

□ Prázdný seznam

- $[] :: [a]$

□ Neprázdný seznam

- $1:2:3:[] \sim [1,2,3] :: [\text{Int}]$
- $(x:xs)$



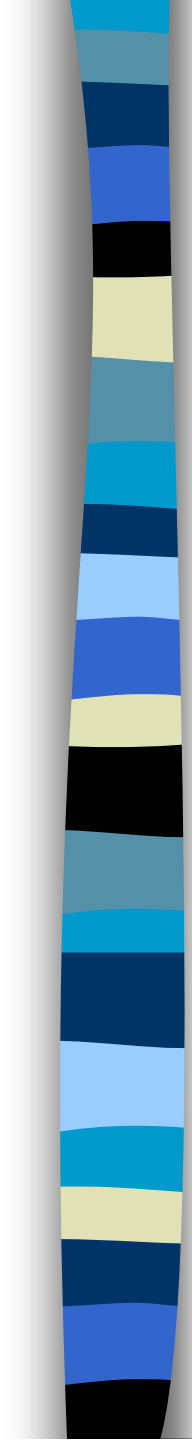
N-tice

□ Konstruktor

- ,
- (a,b,c,d, ...)

□ Příklad

- $(1, 2) :: (\text{Int}, \text{Int})$
- $(1, ['a', 'b']) :: (\text{Int}, [\text{Char}])$



Funkce

□ Typ

- $f :: a \rightarrow b$

□ Signatura typu

- $1 :: \text{Int}$
- $(1, ['a', 'b']) :: (\text{Int}, [\text{Char}])$

Deklarace funkcí

□ Využití

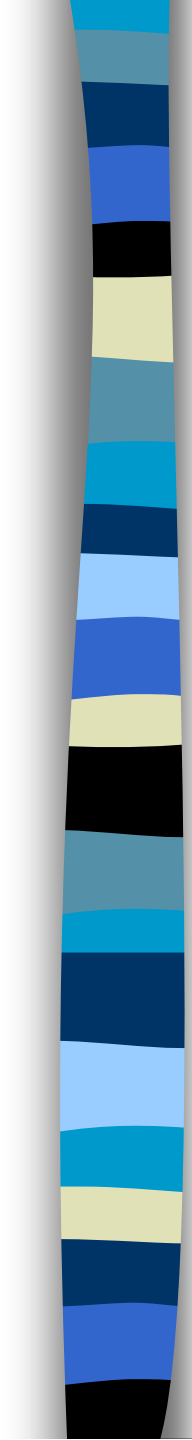
- *Rovnic*

- *Unifikace vzorů (pattern matching)*

□ $f \langle \text{pat11} \rangle \langle \text{pat12} \rangle \dots = \langle \text{rhs1} \rangle$

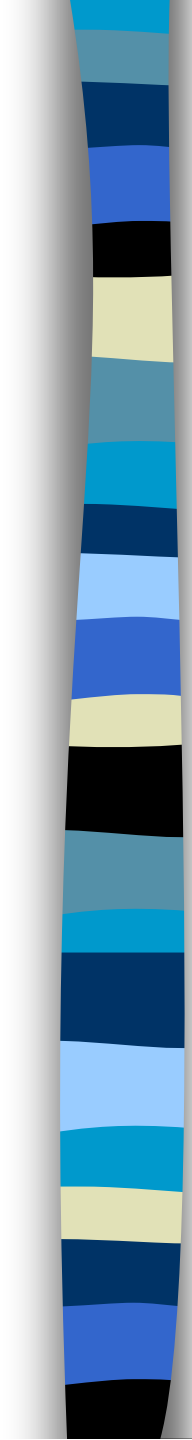
$f \langle \text{pat21} \rangle \langle \text{pat22} \rangle \dots = \langle \text{rhs2} \rangle$

...



Proměnná ve funkci

- `square x = x * x`
- `add x y = x + y`



Konstanta ve funkci

- $\text{not True} = \text{False}$
 $\text{not False} = \text{True}$
- $\text{sgn } 0 = 0$
 $\text{sgn } n = \text{if } n < 0 \text{ then } -1 \text{ else } 1$

Seznam ve funkci

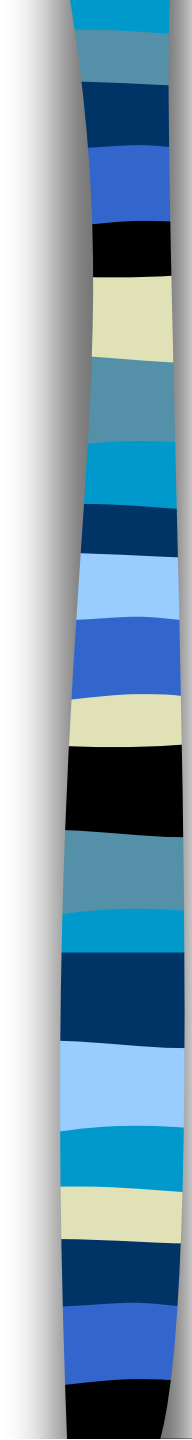
□ $\text{length } [] = 0$

$\text{length } (x:xs) = 1 + \text{length } xs$

□

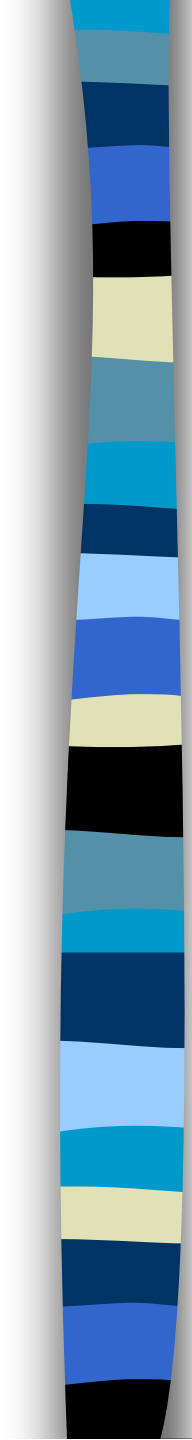
závorky
určují
prioritu

priorita aplikace
je nejvyšší



N-tice ve funkci

- $\text{plus } (x,y) = x+y$
- $\text{swap } (x,y) = (y,x)$



Vzor typu $n+k$ ve funkci

□ $\text{fact } 0 = 1$

$\text{fact } (n+1) = (n+1) * \text{fact } n$

Pojmenování části vzoru ve funkci

- $\text{duphd } p@(x:xs) = x:p$
- $\text{merge } []\ l2 = l2$
 $\text{merge } l1\ [] = l1$
 $\text{merge } l1@(x:xs)\ l2@(y:ys) =$
 if $x < y$
 then $x:\text{merge } xs\ l2$
 else $y:\text{merge } l1\ ys$

Anonymní proměnná ve funkci

- $\text{hd } (x:_) = x$
- $\text{ziphd } (x:_) (y:_) = (x,y)$

Strážené rovnice ve funkci

```

❑ fact n | n < 2      = 1
           | otherwise = n * fact (n-1)

```

```

❑ sgn n | n < 0      = -1
      | n > 0      = 1
      | otherwise    = 0

```


Lokální funkce ve funkci

```
□ sumsqr x y = xx + yy  
    where sqr a b = a*b  
           xx = sqr x x  
           yy = sqr y y
```

Prefixový & infixový zápis funkce

□ $\text{mod} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
 $(+) :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$

□ Infix

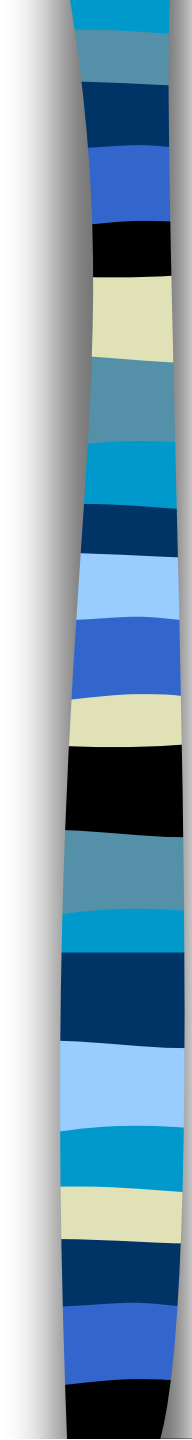
- $5 \text{ `mod` } 3$
 $1 + 1$

□ Prefix

- $\text{mod } 5 \ 3$
 $(+) \ 1 \ 1$

Částečná aplikace funkce

- $\text{inc} = (+) 1$
 - $\text{inc } x = (+) 1 x$
 - $\lambda V. E V = E$
- $\text{inc}' = (1+)$



Odvození typů

- Unifikace – kontrola výskytu
- Známé „pravdy“
 - obecný typ funkce
 - typ konstant
 - typy již odvozené
 - pravidla odvození typu pro danou situaci
- Hledá se nejobecnější typ

Příklad 1

- $f :: a \rightarrow b \rightarrow (a, b)$
 - $(f (3::Int)) :: a \rightarrow (Int, a)$
 - $(f (3::Int) (3.5::Float)) :: (Int, Float)$
- $g :: a \rightarrow a \rightarrow (a, a)$
 - $(\text{if } \dots \text{ then } f \text{ else } g) :: a \rightarrow a \rightarrow (a, a)$
 - $(\backslash x \rightarrow \text{if } \dots \text{ then } f \ x \text{ else } g \text{ „Ahoj“}) ::$
 $\text{String} \rightarrow \text{String} \rightarrow (\text{String}, \text{String})$

Příklad 2

- $f :: a \rightarrow [a]$
 $g :: [a] \rightarrow [a]$
 - (if ... then f else g) :: error
- if f x y then g y x else h (g y)
 - $x :: a$
 - $y :: b$
 - $f :: a \rightarrow b \rightarrow \text{Bool}$
 - $g :: b \rightarrow a \rightarrow c$
 - $h :: (a \rightarrow c) \rightarrow c$

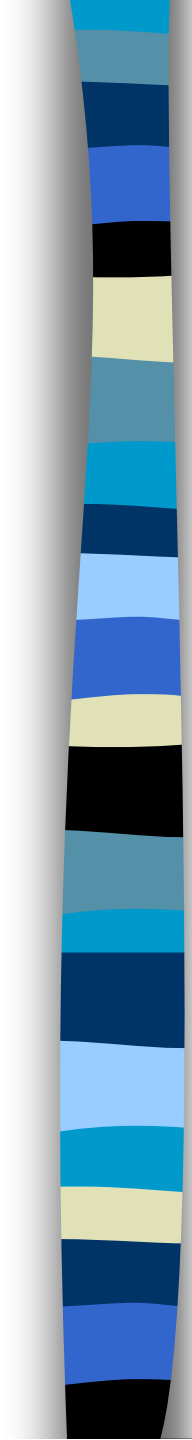
Rekurzivní programování

□ Zpětně rekurzivní funkce

- $f\ x = e(f\ x')$
- po návratu z rekurzivního volání se ještě něco počítá

□ Dopředně rekurzivní funkce

- $f\ x = f(E)$
- rekurzivní volání je poslední část výpočtu



Rekurzivní programování (pokr.)

□ Lineární rekurze

- ve výrazu je jen jedno rekurzivní volání
- lze převést na cyklus

□ Koncová rekurze

- dopředně rekurzivní +lineární
- lze jednoduše přeložit na efektivní cyklus (není třeba uchovávat stav nedokončeného výpočtu)

Rekurzivní programování (pokr.)

□ Příklad

– $\text{rev} :: [a] \rightarrow [a]$

$O(n^2)$

$\text{rev } [] = []$

$\text{rev } (x:xs) = \text{rev } xs ++ [x]$

– $\text{reverse}' :: [a] \rightarrow [a]$

$O(n)$

$\text{reverse}' xs = \text{rev}' xs []$

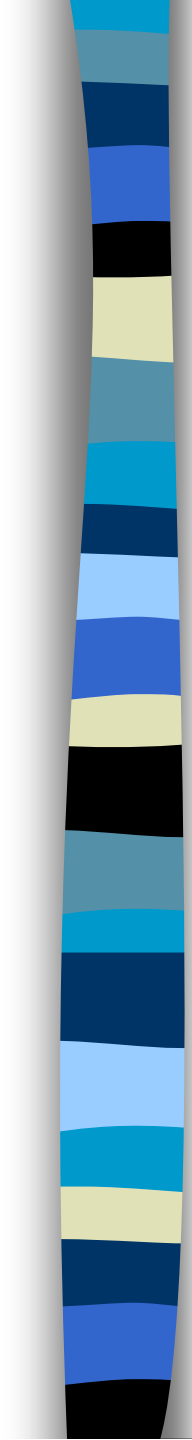
where $\text{rev}' [] ys = ys$

$\text{rev}' (x:xs) ys = \text{rev}' xs (x:ys)$

Funkce pro práci se seznamy

$$\begin{aligned} \square \quad f [] &= \dots \\ f (x:xs) &= \dots \end{aligned}$$

$$\begin{aligned} \square \quad g [x] &= \dots \\ g (x:y:ys) &= \dots \end{aligned}$$



Úkol

- *Vytvořte funkci „sumlist“ pro součet hodnot všech prvků v seznamu celých čísel*
- Jaký je typ funkce?
 - $\text{sumlist} :: [\text{Int}] \rightarrow \text{Int}$
- Jaký je součet prázdného seznamu?
 - $\text{sumlist } [] = 0$

Úkol (pokr.)

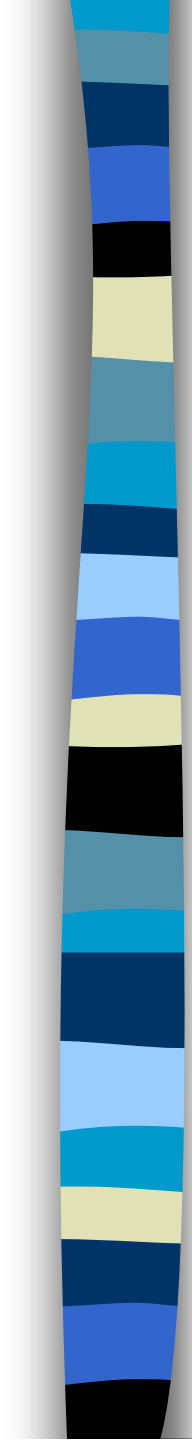
- Jaký je součet neprázdného seznamu (jako funkce součtu seznamu o jeden kratšího)?
 - $\text{sumlist } (x:xs) = x + \text{sumlist } xs$
- Je zaručen konečný počet kroků?
Jestliže ano, potom jak?
 - *Domácí úkol*

Redukce seznamu na hodnotu

- $\text{sumlist} :: [\text{Int}] \rightarrow \text{Int}$
 $\text{sumlist } [] = 0$
 $\text{sumlist } (x:xs) = x + \text{sumlist } xs$
 - $x_1 + (x_2 + (x_3 + \dots))$
- $\text{concat}' :: [[a]] \rightarrow [a]$
 $\text{concat}' [] = []$
 $\text{concat}' (xs:xss) = xs ++ \text{concat}' xss$
 - $xs_1 ++ (xs_2 ++ (xs_3 ++ \dots))$

Změna položek seznamu

- $\text{squareAll} :: [\text{Int}] \rightarrow [\text{Int}]$
 $\text{squareAll } [] = []$
 $\text{squareAll } (x:xs) = (x*x) : \text{squareAll } xs$
 - $x_1*x_1 : (x_2*x_2 : (x_3*x_3 : \dots))$
- $\text{lengthAll} :: [[a]] \rightarrow [\text{Int}]$
 $\text{lengthAll } [] = []$
 $\text{lengthAll } (xs:xss) =$
 $\quad (\text{length } xs) : \text{lengthAll } xss$
 - $\text{length } xs_1 : (\text{length } xs_2 : (\text{length } xs_3 : \dots))$

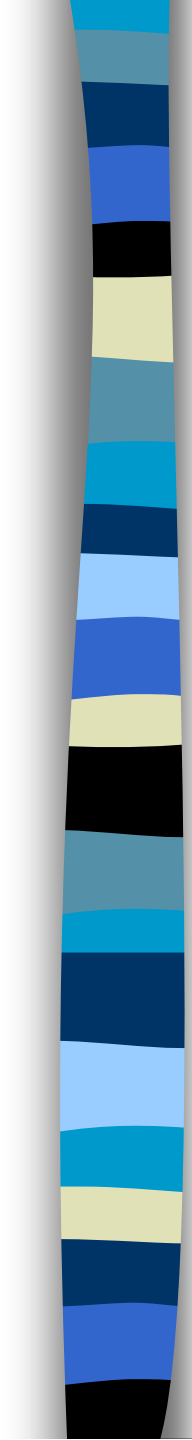


Funkce vyššího řádu

- Co mají funkce společného funkce
 - sumlist a concat'
 - squareAll a lengthAll ?
- Čím se tyto funkce liší?
- Polymorfismus + funkce vyššího řádu
⇒ **abstrakce metody výpočtu**

Abstrakce (1) - folds

- $\text{foldr} :: (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow b$
 $\text{foldr } f \ z \ [] = z$
 $\text{foldr } f \ z \ (x:xs) = f \ x \ (\text{foldr } f \ z \ xs)$
- $\text{foldl} :: (a \rightarrow b \rightarrow a) \rightarrow a \rightarrow [b] \rightarrow a$
 $\text{foldl } f \ z \ [] = z$
 $\text{foldl } f \ z \ (x:xs) = \text{foldl } f \ (f \ z \ x) \ xs$
- $\text{concat'' } xss = \text{foldr } (++) \ [] \ xss$
- $\text{sumlist' } xs = \text{foldr } (+) \ 0 \ xs$



Abstrakce (1) - úkol

- $\text{foldr } (\oplus) \text{ z xs} = \text{foldl } (\oplus) \text{ z xs}$
- KDY ?
- Která varianta je lepší?

Abstrakce (2) - map

- $\text{map} :: (a \rightarrow b) \rightarrow [a] \rightarrow [b]$
 $\text{map } f [] = []$
 $\text{map } f (x:xs) = f x : \text{map } f xs$
- $\text{squareAll}' \text{ } xs = \text{map } sq \text{ } xs$
 where $sq \text{ } x = x * x$
- $\text{lengthAll}' \text{ } xss = \text{map } \text{length} \text{ } xss$

Abstrakce - příklady

- $\text{filter} :: (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$
 $\text{filter } _ [] = []$
 $\text{filter } p (x:xs)$
 - $\quad | p x \quad \quad \quad = x:xs'$
 - $\quad | \text{otherwise} = xs'$ $\quad \text{where } xs' = \text{filter } p \ xs$
- $\text{getEven} :: [\text{Int}] \rightarrow [\text{Int}]$
 $\text{getEven } xs = \text{filter even } xs$

Generátory seznamů

- Matematický zápis definice množiny
 - $\{x \mid x \in \mathbf{N}, \text{even } x\}$
- Zápis v jazyce Haskell
 - `[výraz | kvalifikátory]`

Kvalifikátory

□ Generátor

- $\text{pattern} \leftarrow \text{expression}$
 - vybírá postupně prvky seznamu *expression* a unifikuje je se vzorem *pattern*

□ Filtr

- predikát ($a \rightarrow \text{Bool}$)

Příklady

- $[x \mid x \leftarrow xs] \equiv xs$
- $[f x \mid x \leftarrow xs] \equiv \text{map } f \text{ } xs$
- $[x \mid x \leftarrow xs, p x] \equiv \text{filter } p \text{ } xs$
- $[e \mid x \leftarrow xs, y \leftarrow ys, \dots] \equiv$
 $\text{concat } [[e \mid y \leftarrow ys, \dots] \mid x \leftarrow xs]$

Příklady (pokr.)

- $[d \mid d \leftarrow [1..n], n \text{ `mod` } d == 0] =$
 $[d \mid d \leftarrow \text{filter } (\lambda d \rightarrow n \text{ `mod` } d == 0) [1..n]]$
 $=$
- $\text{filter } (\lambda d \rightarrow n \text{ `mod` } d == 0) [1..n]$

Příklady (pokr.)

- $[(m,n) \mid m \leftarrow [1..3], n \leftarrow [4,5]]$
 $= [(1,4),(1,5),(2,4),(2,5),(3,4),(3,5)]$
- $[' ' \mid n \leftarrow [1..10]] = \text{“} _ _ _ _ _ _ _ _ _ _ \text{”}$
- spaces $n = [' ' \mid i \leftarrow [1..n]]$

Důležité pojmy

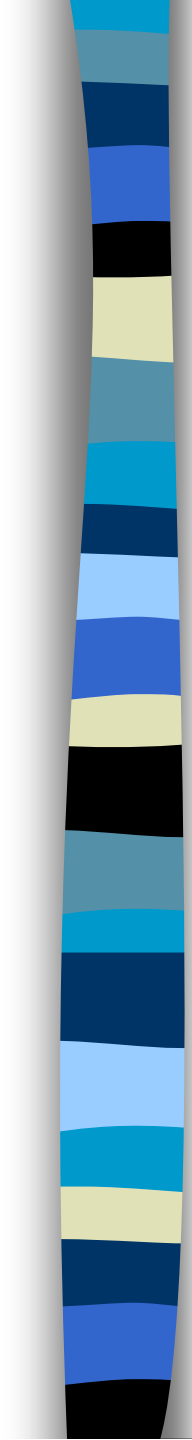
- Typ, jeho signatura
 - Typová proměnná
- Vzor
- Formy rekurze
 - Lineární, koncová, dopředná, zpětná
- Zastavení rekurze, podmínka
- Funkce vyššího řádu
- Generátory seznamů



Úkoly



- Vytvořte funkci, která dostane jako argument seznam a vrátí ho tak, že vždy dvojice prvků bude prohozena
- Bude vaše implementace fungovat i nad nekonečnými seznamy?
- Vytvořte nekonečný seznam všech lichých členů Fibonacciho posloupnosti a ověřte správnost řešení

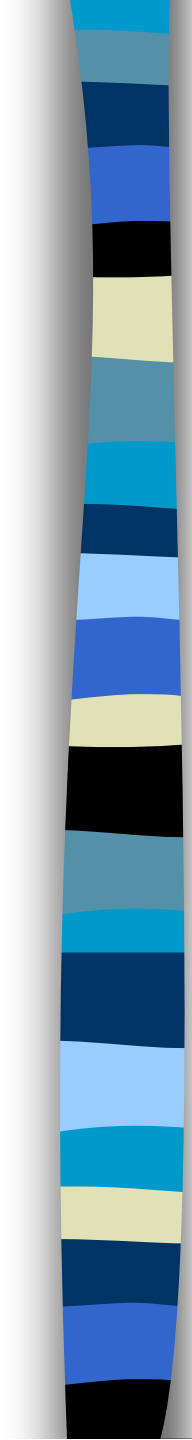


Uživatelské datové typy, typové třídy, pole

Cíle oddílu



- Definice a využití vlastních datových typů
- Typové třídy
 - Definice
 - Užití
 - Význam



Bázové typy

- V každém jazyce
 - `2 :: Int`
 - `'a' :: Char`
 - `3.45 :: Float`
 - *`True :: Bool`*

Základní - odvozené zázázových

□ Běžně v jazyce

– homogenní - seznam

- $[1,2] :: [\text{Int}]$
- $['a','b'] :: [\text{Char}]$
- $[[],[1]] :: [[\text{Int}]]$

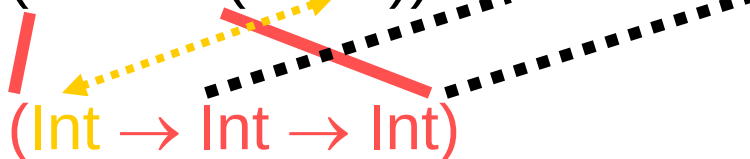
– heterogenní - n-tice

- $(1,'a',5650.4) :: (\text{Int}, \text{Char}, \text{Float})$
- $([23,42],'A') :: ([\text{Int}], \text{Char})$

Monomorfní funkce

- Operátor aplikace funkce $\rightarrow$
 - zprava asociativní

- $\text{addInt} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \sim \text{Int} \rightarrow (\text{Int} \rightarrow \text{Int})$

- $(\text{addInt } (1 :: \text{Int})) :: \text{Int} \rightarrow \text{Int}$

 $(\text{Int} \rightarrow \text{Int} \rightarrow \text{Int})$

Polymorfní funkce

□ Je číslo n v seznamu?

- $\text{elem} :: \text{Int} \rightarrow [\text{Int}] \rightarrow \text{Bool}$
 $\text{elem} _ [] = \text{False}$
 $\text{elem} n (x:xs) = \text{if } n == x$
 then True
 else $\text{elem } n \ xs$

Polymorfní funkce (pokr.)

□ Je znak c v seznamu?

- $\text{elemC} :: \text{Char} \rightarrow [\text{Char}] \rightarrow \text{Bool}$
 $\text{elemC } _ [] = \text{False}$
 $\text{elemC } c (x:xs) = \text{if } c == x$
 then True
 else $\text{elemC } c xs$

Polymorfní funkce (pokr.)

- Je objekt daného typu v seznamu?
(použití algoritmu pro všechny entity)
 - $\text{elem} :: a \rightarrow [a] \rightarrow \text{Bool}$
 $\text{elem } _ [] = \text{False}$
 $\text{elem } y (x:xs) = \text{if } y == x$
 then True
 else elem y xs
 - **a je typová proměnná**

Přetěžování?

- elemI ~ if **n** == x :: Int → Int → Bool
- elemC ~ if **c** == x :: Char → Char ...
- elem ~ if **y** == x :: a → a → Bool

- C, Pascal - přetěžování
- Haskell - typové třídy
 - class Eq a where
 (==) :: a → a → Bool

Typové třídy

- Typ ***a*** je instancí třídy ***C***, je-li pro něj definována vymezená množina operací
 - class Eq a where
$$(==) :: a \rightarrow a \rightarrow \text{Bool}$$
- Potom
 - elem :: (Eq a) => a → [a] → Bool
 - Pro všechny typy a, které jsou instancí třídy Eq, má operace elem typ a → [a] → Bool

Instance typové třídy

- instance Eq Int where
 x==y = intEq x y
 // metoda (definice operace)
- instance (Eq a) => Eq [a] where
 []==[] = True
 (x:xs)==(y:ys) = x==y && xs==ys
 == = False

Nezbytné/odvozené metody

- class Eq a where
 $(==), (/=) :: a \rightarrow a \rightarrow \text{Bool}$
 $x/=y = \text{not } (x==y)$

Dědičnost

□ Jednoduchá

- `class (Eq a) => Ord a where`
 `(<),(<=),(>=),(>) :: a -> a -> Bool`
 `max,min :: a -> a -> a`

□ Vícenásobná

- `class (Eq a, Show a) => C a where`
 ...

Typové třídy vyššího řádu

- $\text{map } f [] = []$
 $\text{map } f (x:xs) = f x : \text{map } f xs$
- `class Functor f where`
 $\text{fmap} :: (a \rightarrow b) \rightarrow f a \rightarrow f b$
- f je označení typu (jméno typu)
- pro případ seznamu jsou to hranaté závorky

Uživatelské datové typy

□ Typová synonyma

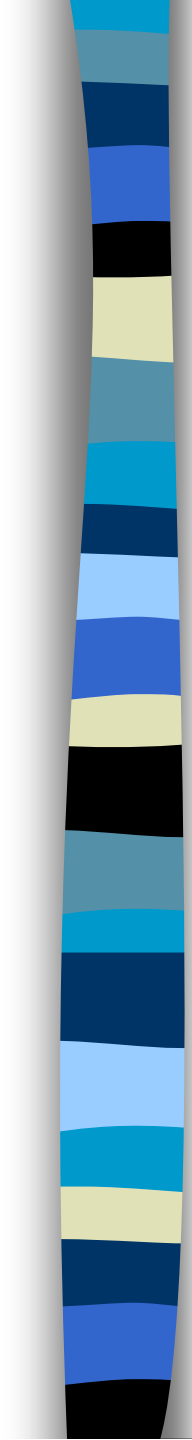
- `type ComplexF = (Float, Float)`
- `type Matrix a = [[a]]`

□ Jednoduché typy

- `newtype ComplexC = ReIm (Float,Float)`
- `newtype MatrixC a = Matrix [[a]]`

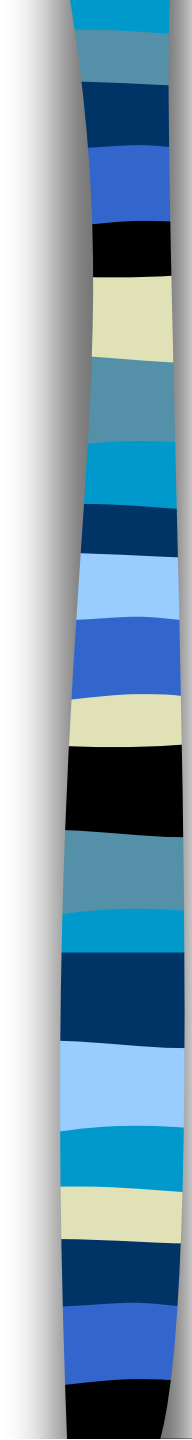
Komplexní datové typy

- **data** Datový_typ $a_1 \dots a_n =$
Constr₁ | ... | Constr_n
 - Datový typ = *jméno typu*
 - a_i = *typové proměnné (nepovinné)*
 - Constr_i = *konstruktory typu*



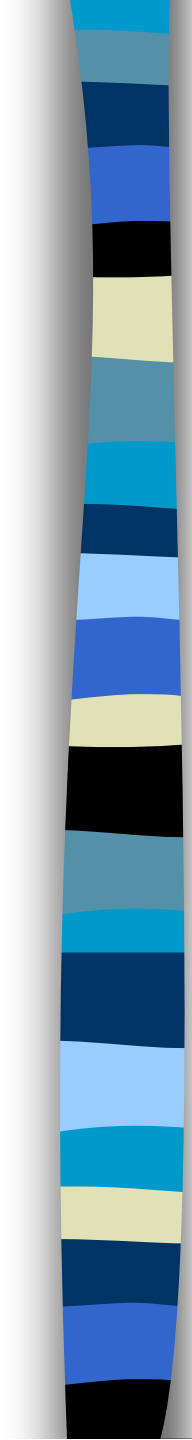
Výčtové typy

- data Color = Red | Green | Blue
 - Red, Green, Blue jsou nulární konstruktory
- isRed :: Color → Bool
 - isRed Red = True
 - isRed _ = False



Rozšířené typy

- data Color' =
 Red |
 Green |
 Blue |
 Grayscale Int
- Grayscale :: Int $\rightarrow$ Color'



Parametrické typy

- data Point a = Pt a a
- (Pt 2 4) :: Point Int
- (Pt 3.4 4.5) :: Point Float
- (Pt (Pt 2 5) (Pt 4 5)) :: Point (Point Int)
- **Pt 4 5.6 - error**

Rekurzivní datové typy

- data Tree a =
 Lf |
 Nd a (Tree a) (Tree a)
- inOrd :: Tree a → [a]
 inOrd Lf = []
 inOrd (Nd x l r) = inOrd l ++ (x:inOrd r)

Datové typy a typové třídy

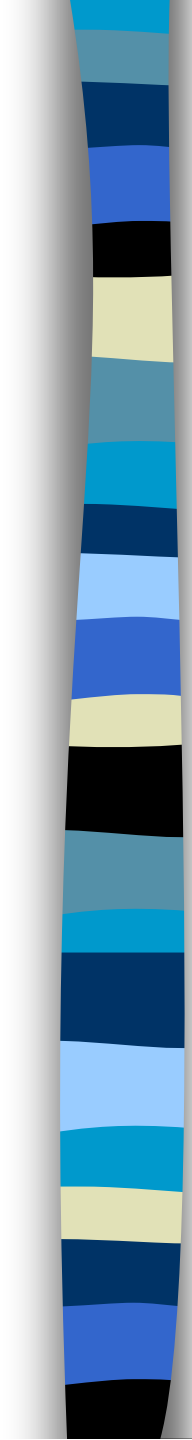
- instance Eq a => Eq (Tree a) where
Lf == Lf = True
(Nd x l1 r1) == (Nd y l2 r2) =
x==y && l1==l2 && r1==r2
== = False
- instance Functor Tree where
fmap _ Lf = Lf
fmap f (Nd x l1 l2)=
Nd (f x)(fmap f l1)(fmap f l2)

Datové typy a typové třídy

- instance Show a => Show (Tree a) where
showsPrec _ Lf = (++) "Lf"
showsPrec _ (Nd a Lf Lf) =
 ("Nd "++).shows a . ((++) " Lf Lf")
showsPrec _ (Nd a Lf r) =
 ("Nd "++).shows a . ((++) " Lf (") . shows r . (')' :)
showsPrec _ (Nd a l Lf) =
 ("Nd "++).shows a . ((++) " (").shows l . ((++) ")" Lf")
showsPrec _ (Nd a l r) =
 ("Nd "++).shows a . ((++) " (") . shows l .
 ((++) ")" (") . shows r . (')' :)

Poznámka

- *showsPrec p t*
 - $p \sim$ precedence
 - $t \sim$ typ
- *p udává prioritu a podle toho řadí do tisku závorky či nikoliv*
 - vyšší p tím vyšší je priorita ,kolem', tj. je třeba to, co převádím, dát do závorek
- *podobně existuje readsPrec pro třídu Read*



Pro/proti vlastního přístupu

□ Pro

- vše je pod kontrolou
- možno zavést instanci i u složitých typů

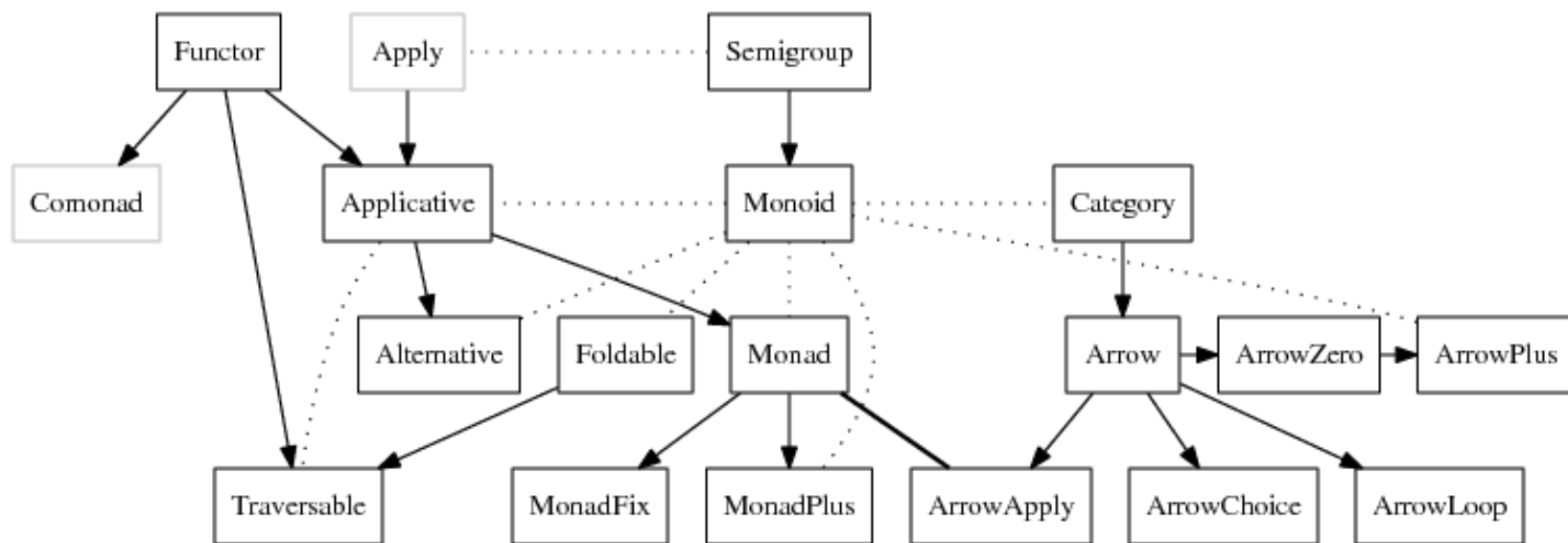
□ Proti

- často pracné
- implementace s využitím priorit není triviální
- klíčové slovo **deriving**

Použití deriving

- `data Color = Red | Green | Blue`
`deriving (Eq,Ord,Enum,Bounded,Show,Read)`
- `data Tree a =`
 `Lf |`
 `Nd a (Tree a) (Tree a)`
`deriving (Eq,Show)`

Přehled tříd v Haskellu – příklad!



Pole

□ Indexy

- class (Ord a) => Ix a where
 range :: (a,a) → [a]
 index :: (a,a) → a → Int
 inRange :: (a,a) → a → Bool

- range (0,4) ~> [0,1,2,3,4]
 range ((0,0),(1,2)) ~>
 [(0,0),(0,1),(0,2),(1,0),(1,1),(1,2)]
- index (1,9) 2 ~> 1
 index ((0,0),(1,2)) (1,1) ~> 4

Vytvoření pole

□ Vytvoření monolitického pole

- $\text{array} :: (\text{Ix } a) \Rightarrow (a,a) \rightarrow [(a,b)] \rightarrow \text{Array } a \ b$
 - spodní/horní limit
 - inicializace pole, páry index hodnota

□ Příklad

- $\text{squares} = \text{array } (1,100) [(i,i*i) \mid i \leftarrow [1..100]]$
- $\text{squares}!7 \rightsquigarrow 49$
- $\text{bounds squares} \rightsquigarrow (1,100)$

Pole s kumulovanými hodnotami

□ Funkce pro vytvoření

□ `accumArray :: (Ix a) =>`

`(b → c → b) → b → (a,a) → [Assoc a c] → Array a b`

– kumulační funkce

– inicializační hodnota (pro každou buňku stejná)

– spodní/horní limit pole

– seznam asociací (index-hodnota)

□ Histogram

– `hist bnds is = accumArray (+) 0 bnds [(i,1) | i ← is, inRange bnds i]`

Změny hodnot prvků

□ Operátor změny

- $(//) :: (lx\ a) \Rightarrow \text{Array } a\ b \rightarrow [(a,b)] \rightarrow \text{Array } a\ b$

□ Příklad (záměna řádků matice)

□ `swapRows i i' a =`

`a // ([((i , j), a!(i' , j)) | j ← [jLo..jHi]] ++
[((i' , j), a!(i , j)) | j ← [jLo..jHi]])`

`where ((iLo, jLo), (iHi, jHi)) = bounds a`

Důležité pojmy

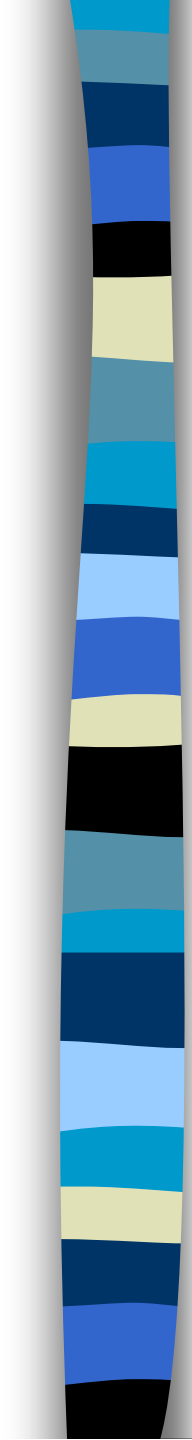


- Polymorfismus, přetěžování
- Typová třída, instance
- Dědičnost v typových třídách
- Typy:
 - Synonyma, jednoduché, **komplexní**
 - Výčtové, rozšířené, parametrické, rekurzivní

Úkoly



- Navrhněte vlastní systém tříd a operátorů práci s celými a desetinnými čísly tak, aby operace $3+4,5$ nevedla na implicitní konverzi
- Implementujte a ověřte funkčnost toho systému
- Navrhněte a implementujte třídu pro práci s vyhledávacími stromy

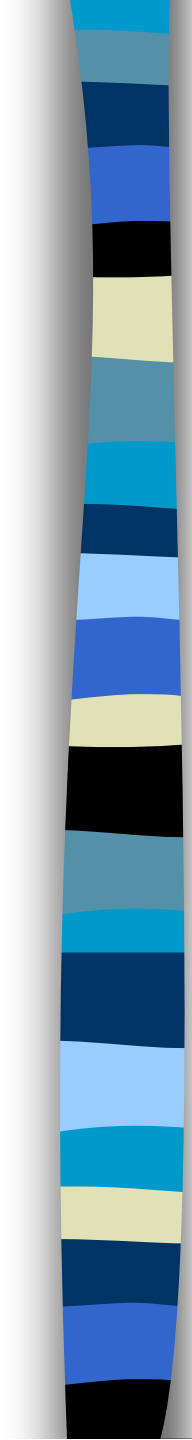


Vstupy/výstupy

Cíle oddílu

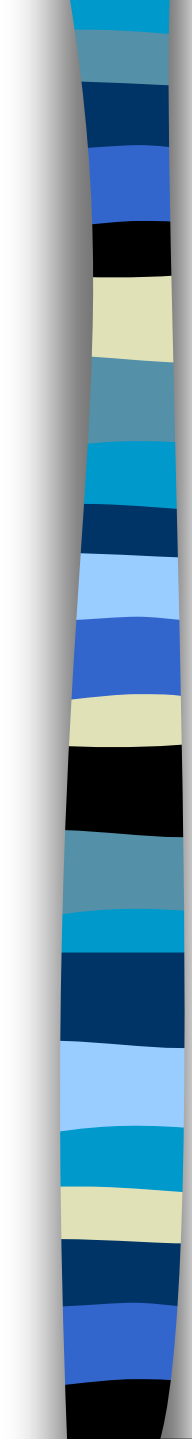


- Práce s monadickými třídami
- V/V operace jako monadické třídy



Monadické třídy

- Základní třídy spojené s monádami (nikoliv monoidy)
 - Functor, Monad, MonadPlus
- Členové těchto tříd
 - IO, seznamy (`[]`), výjimky (Maybe)
 - výjimky jsou zkrácené seznamy
Nothing $\sim []$
Just x $\sim [x]$



Functor

- Definuje funkci *fmap*, která aplikuje jednu operaci na všechny elementy struktury beze změny tvaru struktury
 - $\text{fmap id} = \text{id}$
 $\text{fmap } (f . g) = \text{fmap } f . \text{fmap } g$

Monad

- `class Applicative m => Monad (m :: * -> *)`
where
 - `(>>=) :: m a -> (a -> m b) -> m b`
 - `(>>) :: m a -> m b -> m b`
 - `return :: a -> m a`
 - `fail :: String -> m a`
`m >> k = m >>= (\_ -> k)`

Syntaktické ekvivalenty

- $[(x,y) \mid x \leftarrow [1,2,3], y \leftarrow [4,5,6]]$
- ```
do x ← [1,2,3]
 y ← [4,5,6]
 return (x,y)
```
- $[1,2,3] \gg = (\backslash x \rightarrow [4,5,6] \gg = (\backslash y \rightarrow \text{return } (x,y)))$



# Sémantické ekvivalenty

- $\text{return } a \gg= k \quad = k \ a$
- $m \gg= \text{return} \quad = m$
- $\text{map } f \ xs \quad = xs \gg= \text{return} . f$
- $m \gg= (\lambda x \rightarrow k \ x \gg= h) \quad = (m \gg= k) \gg= h$

# MonadPlus

## □ Definice

- `class (Alternative m, Monad m)`  
    `=> MonadPlus (m :: * -> *)` where  
    `mzero`           `:: m a`  
    `mplus`           `:: m a -> m a -> m a`

## □ Vlastnosti

- `mzero >>= f = mzero`
- `v >> mzero = mzero`
- `mplus = (<|>)`
- `(mzero :: [Int]) `mplus` [1,2,3] =`  
    `[1,2,3] `mplus` (mzero :: [Int]) = [1,2,3]`

# Akce ve FP

- V imperativních jazycích je program tvořen posloupností *akcí* (čtení a nastavování globálních proměnných, čtení a zápis souborů)
- Haskell: oddělení akcí od čistě funkcionálního kódu pomocí *monadických operátorů*
- Akce je funkce s výsledkem typu **IO** a

# Příklady akcí

- `getChar :: IO Char`                      `-- akce vracející znak`
- `putChar :: Char → IO ()`                `-- akce nevracející nic`
- `return :: a → IO a`                      `-- hodnota => akce`
- `ready :: IO Bool`  
    `ready = do c ← getChar`  
            `return (c=='y')`                      `-- !! return !!`

# Funkce main

- představuje hlavní program; je to akce, která nic nevrací

```
– main :: IO ()
 main = do c ← getChar
 putChar c
```

# Čtení řádku textu

- Přečteme znak, je-li to konec řádku, vrátíme prázdný řetězec; jinak přečteme zbytek řádku, spojíme ho s načteným znakem a vrátíme
  - `getLine :: IO String` -- akce vracející řetězec  
`getLine = do x ← getChar`  
    if `x == '\n'` then return ""  
    else do `xs ← getLine`  
            return (x:xs)

# Vypsání řetězce

- Na všechny řetězce aplikujeme funkci *putChar*, např. funkcí *map*
  - *map putChar s* je typu `[IO()]`, tj. seznam akcí => **není to akce**
  - je třeba ještě použít funkci
    - `sequence :: Monad m => [m a] → m [a]`
  - `putStr :: String → IO()`  
`putStr s = sequence (map putChar s)`

# Výjimky

- Datový typ a konstruktor *GHC.IO.Exception*
  - **IOError** ~ `IOException`
    - `ioe_type` – položka
  - `IOErrorType`
    - `EOF`, `NoSuchThing`, ...



# Výjimky

- Ke každé výjimce existuje funkce *System.IO.Error*
  - **isXXXError :: IOError → Bool**
    - isEOFError

# Zachycení výjimky

- Výjimku zachytává funkce *catch*
  - $\text{catch} :: \text{IO } a \rightarrow (\text{IOError} \rightarrow \text{IO } a) \rightarrow \text{IO } a$ 
    - první parametr je akce, během které se mají výjimky odchyťovat
    - druhý parametr je funkce, která se zavolá, pokud nastane výjimka - funkce musí vrátit náhradní výsledek
- ! *catch* definováno 2x !
  - Prelude (pro IO) x Exception

# Generování výjimky

- Výjimku akce generuje funkce *ioError*
  - $\text{ioError} :: \text{IOError} \rightarrow \text{IO a}$   
*System.IO.Error*  
*IO*
- Výsledek funkce *ioError* je kompatibilní s **libovolnou akcí** (nezáleží na vrácené hodnotě, typ se přizpůsobí kontextu)

# Příklad č. 1

## □ Čtení znaku s ignorováním chyb

- `getChar` = `getChar` 'catch' (`\_ → return '\n'`)
- pokud při čtení nastane chyba (např. konec souboru), vrátí se znak konce řádku
- neodlišuje konec souboru od ostatních chyb

## Příklad č. 2

### □ Čtení znaku s rozlišením konce souboru

- `getChar' = getChar 'catch' eofHandler`  
    where `eofHandler e =`  
        if `isEOFError e` then return `'\n'`  
        else `ioError e`
- pokud se při čtení dostaneme na konec souboru, vrací se znak konce řádku, jinak se vzniklá výjimka vygeneruje znovu

# Práce se soubory

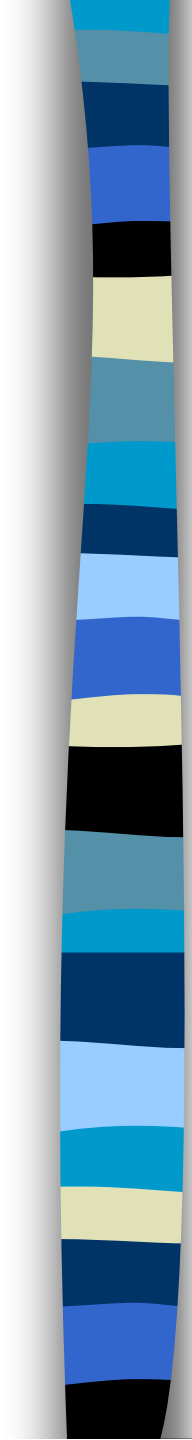
## □ Otevření a uzavření souboru

- `type FilePath = String`
- `data IOMode = ReadMode | WriteMode | AppendMode | ReadWriteMode`
- `openFile :: FilePath → IOMode → IO Handle`
- `hClose :: Handle → IO ()`

# Práce se soubory

## □ Čtení ze souboru

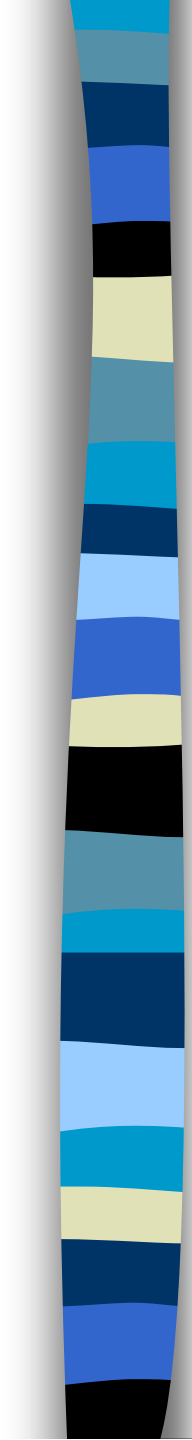
- `hGetChar :: Handle → IO Char`
- `stdin, stdout, stderr :: Handle`
- `getChar = hGetChar stdin`
- `hGetContents :: Handle → IO String`



# Práce se soubory

- Funkce začínající na ,h‘ dostávají jako první parametr handle na otevřený soubor, varianty bez ,h‘ pracují se standardními soubory
- Funkce *hGetContents* (jako i ostatní) se vyhodnocuje ,lazy‘, tj. skutečný vstup dat je požadován až tehdy, když se výsledná hodnota začne vyhodnocovat





# Příklad

- Program pro kopírování souboru
- Funkce *opf* se zeptá na jméno vstupního, resp. výstupního souboru a tento soubor otevře v odpovídajícím režimu
- Pokud se soubor nepodaří otevřít, opakuje se dotaz na jméno

# Příklad (pokr.)

```
import IO
```

```
main = do fromHandle ← opf "Copy from: " ReadMode
 toHandle ← opf "Copy to: " WriteMode
 contents ← hGetContents fromHandle
 hPutStr toHandle contents
 hClose toHandle
 hClose fromHandle
 putStr "Done"
```

# Příklad (pokr.)

`opf :: String → IO Mode → IO Handle`

`opf prompt mode =`

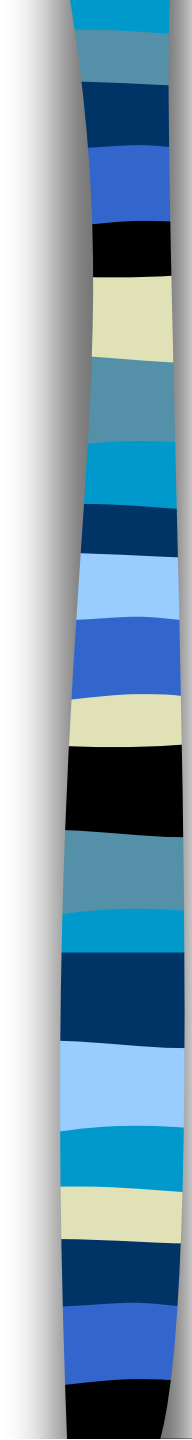
`do putStr prompt`

`name ← getLine`

`catch (openFile name mode)`

`(\_ → do putStr ("Can't open " ++ name ++ "\n")`

`opf prompt mode)`



# Příklad – nová varianta

- ❑ Program pro kopírování souboru – funkce `readFile`
- ❑ Funkce *oprř* se zeptá na jméno vstupního souboru a vrátí jeho obsah
- ❑ Funkce *opwř* se zeptá na jméno výstupního souboru a tento soubor otevře v odpovídajícím režimu
- ❑ Pokud se soubor nepodaří otevřít, opakuje se dotaz na jméno

# Příklad – nová varianta (pokr.)

```
import IO
```

```
main = do fromCts ← oprf "Copy from: "
 toHandle ← opwf "Copy to: "
 hPutStr toHandle fromCts
 hClose toHandle
 putStr "Done"
```

# Příklad – nová varianta (pokr.)

`opwf :: String → IO Handle`

`opwf prompt =`

`do putStr prompt`

`name ← getLine`

`catch (openFile name WriteMode)`

`(\_ → do putStr ("Can't open " ++ name ++ "\n")`

`opwf prompt)`

# Příklad – nová varianta (pokr.)

`opr :: String → IO Handle`

`opr prompt =`

`do putStr prompt`

`name ← getLine`

`catch (readFile name)`

`(\_ → do putStr ("Can't open \"++name++\"\n")`  
                `opr prompt)`

`Control.Exception`

`Control.Exception.IOException`

# Case study - kalkulátor

## □ Zadání:

- Sestrojte program, který čte ze vstupu jednoduché výrazy (i přiřazovací) a hned je vyhodnocuje.
  - paměť
  - *parser - příští přednáška*
  - zpracování vstupu



# Výrazy

- module Expr  
    (Expr(Lit, Var, Op),  
    Ops(Add, Mul, Sub, Div, Mod),  
    Var) where

data Expr = Lit Int | Var Var | Op Ops Expr Expr

data Ops = Add | Sub | Mul | Div | Mod

type Var = Char

# Paměť

- module Store  
    ( Store, initial, value, update ) where

import Expr

newtype Store = Sto (Var  $\rightarrow$  Int)

initial :: Store

initial = Sto ( $\backslash v \rightarrow 0$ )

# Paměť (pokr.)

- $\text{value} :: \text{Store} \rightarrow \text{Var} \rightarrow \text{Int}$   
 $\text{value} (\text{Sto sto}) v = \text{sto } v$

$\text{update} :: \text{Store} \rightarrow \text{Var} \rightarrow \text{Int} \rightarrow \text{Store}$   
 $\text{update} (\text{Sto sto}) v n$   
 $= \text{Sto } (\backslash w \rightarrow \text{if } v == w \text{ then } n \text{ else } \text{sto } w)$

# Parser

- module Parser  
    (Command(Eval, Assign, Null),  
      commLine) where

import Expr

data Command = Eval Expr | Assign Var Expr |  
              Null

commLine :: String → Command

# Kalkulátor

- module Calc(mainCalc) where  
import Expr  
import Store  
import Parser

```
while :: IO Bool -> (a -> IO a) -> a -> IO ()
while test action initarg = do
 res <- test
 if res
 then action initarg >>=
 while test action
 else return ()
```

# Kalkulátor (pokr.)

- $\text{eval} :: \text{Expr} \rightarrow \text{Store} \rightarrow \text{Int}$   
     $\text{eval} (\text{Lit } n) \_ = n$   
     $\text{eval} (\text{Var } v) \text{ st} = \text{value st } v$   
     $\text{eval} (\text{Op op } e1 \ e2) \text{ st} = \text{opVal op } v1 \ v2$   
        where  $v1 = \text{eval } e1 \ \text{st}$   
               $v2 = \text{eval } e2 \ \text{st}$   
     $\text{opVal Add } v1 \ v2 = v1 + v2$   
     $\text{opVal Mul } v1 \ v2 = v1 * v2$   
     $\text{opVal Sub } v1 \ v2 = v1 - v2$   
     $\text{opVal Div } v1 \ v2 = v1 \ `div` v2$   
     $\text{opVal Mod } v1 \ v2 = v1 \ `mod` v2$

# Kalkulátor (pokr.)

- $\text{command} :: \text{Command} \rightarrow \text{Store} \rightarrow (\text{Int}, \text{Store})$   
   $\text{command Null st} = (0, \text{st})$   
   $\text{command (Eval e) st} = (\text{eval e st}, \text{st})$   
   $\text{command (Assign v e) st} = (\text{val}, \text{newSt})$   
    where  $\text{val} = \text{eval e st}$   
           $\text{newSt} = \text{update st v val}$

# Kalkulátor (pokr.)

- $\text{calcStep} :: \text{Store} \rightarrow \text{IO Store}$   
calcStep st =  
    do line  $\leftarrow$  getLine  
        let comm = commLine line  
            (val, newSt) = command comm st  
        print val  
        return newSt



# Kalkulátor (pokr.)

- `calcSteps :: Store → IO ()`  
 `calcSteps st =`  
 `while notEOF calcStep st`

`notEOF :: IO Bool`  
`notEOF = do res ← isEOF`  
          `return (not res)`

`mainCalc = calcSteps initial`

# Důležité pojmy

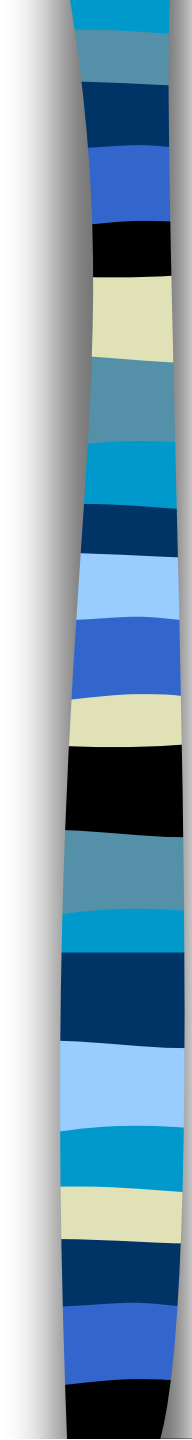
- Monadická třída
- Akce
- Výjimka



# Úkoly



- Prostudujte možnosti práce s binárními soubory
- Navrhňte diskovou i paměťovou reprezentaci pro jednoduchý DB systém v Haskellu
- Implementujte jej, zaměřte se na V/V operace, indexaci DB apod.

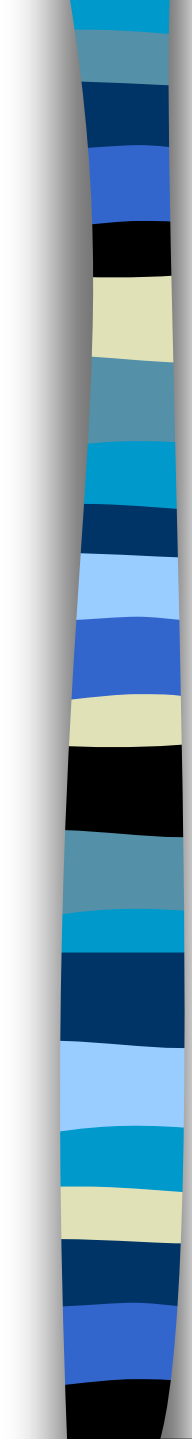


# Praktické úlohy

# Cíle oddílu

- Ukázka práce v jazyku Haskell





# Reprezentace relační databáze

- Navrhněte reprezentaci relační databáze a implementujte operace pro dotazování

# Tabulky

| Author     |             |
|------------|-------------|
| <i>ssn</i> | <i>name</i> |
| 1          | Čapek       |
| 2          | Němcová     |
| 3          | Clarke      |

| Book        |                       |
|-------------|-----------------------|
| <i>isbn</i> | <i>title</i>          |
| 100         | R.U.R.                |
| 101         | Válka s mloky         |
| 102         | Babička               |
| 103         | 2001: Vesmírná odysea |
| 104         | Rajské fontány        |
| 105         | Setkání s Rámou       |

| Wrote      |             |
|------------|-------------|
| <i>ssn</i> | <i>isbn</i> |
| 1          | 100         |
| 1          | 101         |
| 2          | 102         |
| 3          | 103         |
| 3          | 104         |
| 3          | 105         |

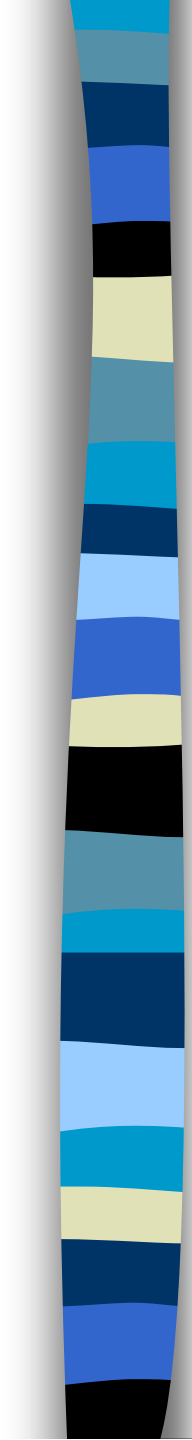
# Návrh reprezentace DB

- Data v tabulce mohou být řetězce, čísla, logické hodnoty nebo nedefinované hodnoty

- data Attribute = 

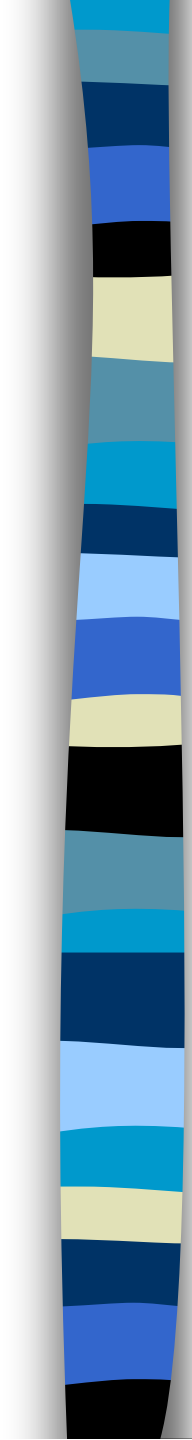
|      |        |
|------|--------|
| St   | String |
| Num  | Int    |
| Bool | Bool   |
| Null |        |





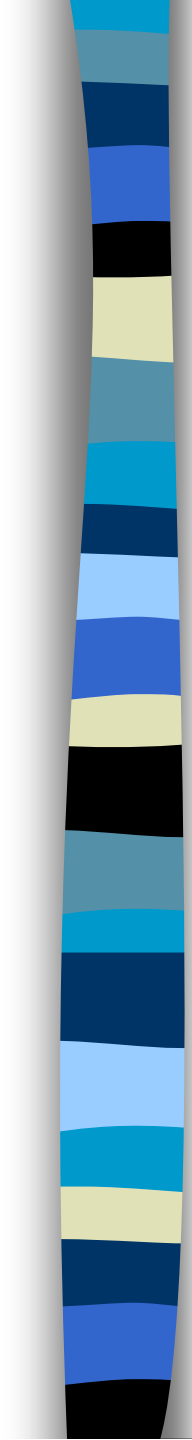
# Návrh reprezentace DB (pokr.)

- Řádek tabulky je tvořen seznamem hodnot
  - type Tuple = [Attribute]



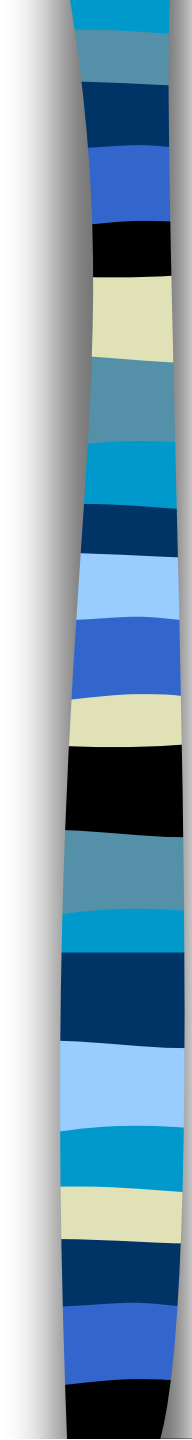
# Návrh reprezentace DB (pokr.)

- Struktura (schéma) tabulky je dána jmény sloupců
  - type Schema = [String]



# Návrh reprezentace DB (pokr.)

- Tabulka je tvořena schématem a seznamem řádků
  - type Table = (Scheme, [Tuple])



# Návrh reprezentace DB (pokr.)

- Databáze je seznam pojmenovaných tabulek
  - `type Database = [(String, Table)]`

# Příklad databáze

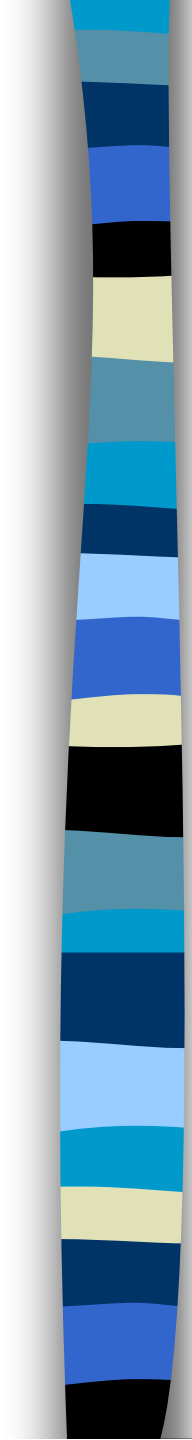
- authors :: Table  
authors = ( ["ssn", "name"],  
            [[Num 1, St "Capek"],  
            [Num 2, St "Nemcova"],  
            [Num 3, St "Clarke"]] )

# Příklad databáze (pokr.)

- books :: Table  
books = ( ["isbn", "title"],  
[[Num 100, St "R.U.R."],  
[Num 101, St "Valka s mloky"],  
[Num 102, St "Babicka"],  
[Num 103, St "2001: Vesmirna ..."],  
[Num 104, St "Rajske fontany"],  
[Num 105, St "Setkani s Ramou"]])

# Příklad databáze (pokr.)

- wrote :: Table  
wrote = ( ["ssn", "isbn"],  
[[Num 1, Num 100],  
[Num 1, Num 101],  
[Num 2, Num 102],  
[Num 3, Num 103],  
[Num 3, Num 104],  
[Num 3, Num 105]] )



# Příklad databáze (pokr.)

- `db :: Database`  
`db = [ ("Author", authors),`  
      `("Book", books),`  
      `("Wrote", wrote) ]`



# Návrh reprezentace dotazu

- Dotaz je výraz nad jmény tabulek s operátory sjednocení, rozdíl, kartézský součin, projekce, selekce, join
  - data Query = Table String
    - | Union Query Query
    - | Difference Query Query
    - | Cross Query Query
    - | Project Schema Query
    - | Select Condition Query
    - | Join Query Query
    - ...

# Návrh reprezentace dotazu (pokr.)

- Podmínka je logický výraz s operátory AND, OR, NOT a relačními operátory
  - data Condition = NOT Condition
    - | AND Condition Condition
    - | OR Condition Condition
    - | EQU Expr Expr
    - | NEQ Expr Expr
    - ...

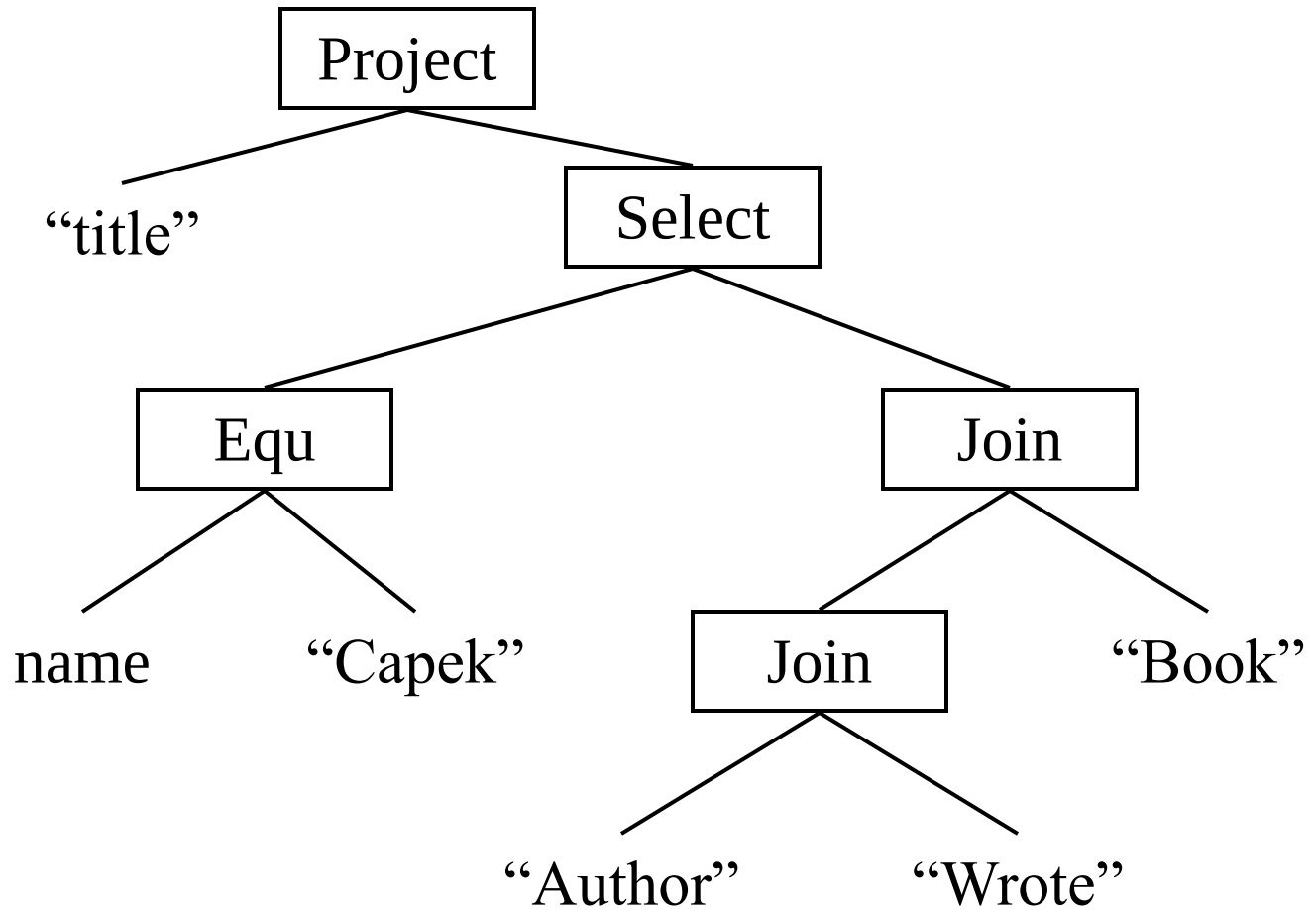
# Návrh reprezentace dotazu (pokr.)

- Aritmetický výraz je tvořen jmény atributů, konstantami a aritmetickými operátory
  - data Expr = Attr String  
| Const Attribute  
| Plus Expr Expr  
| Minus Expr Expr  
...

# Příklad dotazu

- SELECT title  
FROM author JOIN wrote JOIN book  
WHERE name="Capek"
- qry :: Query  
  qry :: Project ["title"]  
    (Select (EQU (Attr "name")  
              (Const (St "Capek"))))  
    (Join (Join (Table "Author")  
              (Table "Wrote"))  
          (Table "Book")))

# Reprezentace dotazu



# Realizace funkcí pro operátory

- $\text{union} :: \text{Table} \rightarrow \text{Table} \rightarrow \text{Table}$   
 $\text{cross} :: \text{Table} \rightarrow \text{Table} \rightarrow \text{Table}$   
 $\text{difference} :: \text{Table} \rightarrow \text{Table} \rightarrow \text{Table}$   
 $\text{join} :: \text{Table} \rightarrow \text{Table} \rightarrow \text{Table}$   
 $\text{project} :: \text{Schema} \rightarrow \text{Table} \rightarrow \text{Table}$   
 $\text{select} :: (\text{Schema} \rightarrow \text{Tuple} \rightarrow \text{Bool})$   
 $\quad \rightarrow \text{Table} \rightarrow \text{Table}$

# Sjednocení relací

- Schémata obou relací musí být totožná
- Výsledkem je seznam řádků obsahující řádky z obou tabulek
  - `union (atts1, tuples1) (atts2, tuples2) =  
 (atts1,  
 if atts1==atts2 then tuples1 ++ tuples2  
 else error "union: incompatible relation")`

# Interpretace dotazu

- Vstupem je dotaz a databáze, výstupem tabulka reprezentující výsledek dotazu
- Vyhodnocení post-order průchodem stromu dotazu
  - $\text{retrieve} = \text{Query} \rightarrow \text{Database} \rightarrow \text{Table}$   
 $\text{retrieve query db} = \text{ret query}$   
where  $\text{ret (Union } q1 \ q2) = \text{union (ret } q1) \ (\text{ret } q2)$   
 $\text{ret (Cross } q1 \ q2) = \text{cross (ret } q1) \ (\text{ret } q2)$   
 $\text{ret (Table } t) = \dots$



# Překlad výrazu

- `type Var = Char`
- `data Expr = Lit Int | Var Var | Op Ops Expr Expr`  
deriving (Show,Read,Eq)
- `data Ops = Add | Sub | Mul | Div | Mod`  
deriving (Enum,Bounded,Eq,Show,Read)
- `2 + 3 ~>`  
`Op Add (Lit 2) (Lit 3)`

# Typy překladačů 1

- type Parse1 a b = [a]  $\rightarrow$  b
- bracket “(xyz”  $\sim$ > ‘(‘
- number “234”  $\sim$ > 2 či 23 či 234 ?
- bracket “234”  $\sim$ > ???

# Typy překladačů 2

- type Parse2 a b = [a] → [b]
- bracket “(xyz” ~> [‘(‘]
- number “234” ~> [2, 23, 234]
- bracket “234” ~> []

# Typy překladačů 3

- type Parse a b = [a]  $\rightarrow$  [(b, [a])]
- bracket "(xyz"  $\sim\>$  [(('(', "xyz")]
- number "234"  $\sim\>$  [(2,"34"), (23,"4"), (234,"")]
- bracket "234"  $\sim\>$  []

# Některé základní překladače

- `none :: Parse a b`  
`none inp = []`
- `succeed :: b → Parse a b`  
`succeed val inp = [(val,inp)]`
- `token :: Eq a => a → Parse a a`  
`token t (x:xs)`  
    | `t==x`                    `= [(t,xs)]`  
    | `otherwise`               `= []`  
`token t []`                    `= []`

# Některé základní překladače (pokr.)

- $\text{spot} :: (a \rightarrow \text{Bool}) \rightarrow \text{Parse } a$   
   $\text{spot } p \ (x:xs)$   
     $\quad | \ p \ x \quad \quad = [(x,xs)]$   
     $\quad | \ \text{otherwise} \quad = []$   
   $\text{spot } \_ \ [] \quad \quad = []$
- $\text{bracket} = \text{token '('}$
- $\text{dig} = \text{spot isDigit}$
- $\text{token } t = \text{spot } (==t)$

# Spojování základních překladačů

- $\text{alt} :: \text{Parse } a \rightarrow \text{Parse } b \rightarrow \text{Parse } a \rightarrow b$   
 $\text{alt } p1 \ p2 \ \text{inp} = p1 \ \text{inp} ++ p2 \ \text{inp}$
- infixr 5  $>^*>$   
 $(>^*>) :: \text{Parse } a \rightarrow \text{Parse } b \rightarrow \text{Parse } a \rightarrow (b,c)$   
 $(>^*>) \ p1 \ p2 \ \text{inp}$   
 $= [ ((y,z), \text{rem2}) \mid (y, \text{rem1}) \leftarrow p1 \ \text{inp},$   
 $(z, \text{rem2}) \leftarrow p2 \ \text{rem1} ]$

# Příklad

- (bracket `alt` dig) "234"  
~> [] ++ [(2, "34")]  
~> [(2, "34")]
- number "24("  
~> [(2, "4(", (24, "(")]  
bracket "  
~> [("'", ")]  
(number >\*> bracket) "24("  
~> [ ((24, '(', ")]



# Výsledek překladu

- $\text{build} :: \text{Parse } a \ b \rightarrow (b \rightarrow c) \rightarrow \text{Parse } a \ c$   
 $\text{build } p \ f \ \text{inp} = [(f \ x, \text{rem}) \mid (x, \text{rem}) \leftarrow p \ \text{inp}]$
- $(\text{number } \text{'build'} \ \text{digsToNum}) \text{"21a"}$   
 $\sim> [(2, \text{"1a3"}), (21, \text{"a3"})]$
- $\text{list} :: \text{Parse } a \ b \rightarrow \text{Parse } a \ [b]$   
 $\text{list } p = (\text{succeed } []) \text{'alt'}$   
 $((p \ >^* > \text{list } p) \text{'build'} \ (\text{uncurry } ()))$

# Další užitečné ‘kombinátory’

- `neList :: Parse a b → Parse a [b]`  
`neList p x = filter isEmpty (list p x)`  
    where `isEmpty :: ([a],b) -> Bool`  
          `isEmpty ((_:_),_) = True`  
          `isEmpty _ = False`
- `optional :: Parse a b → Parse a [b]`  
`optional p = (p `build` (\x -> [x])) `alt``  
          `(succeed [])`

# Překladač výrazů

- `parser :: Parse Char Expr`  
`parser = litParse `alt` varParse`  
``alt` opExprParse`
- `varParse :: Parse Char Expr`  
`varParse = spot isVar `build` Var`  
`where isVar :: Char → Bool`  
`isVar x = 'a' <= x && x <= 'z'`

# Překladač výrazů (pokr.)

- $\text{opExprParse} :: [\text{Char}] \rightarrow [(\text{Expr}, [\text{Char}])]$   
 $\text{opExprParse} =$   
    (token '('           >\*>  
      parser           >\*>  
      spot isOp       >\*>  
      parser           >\*>  
      token ')' )  
    `build` makeExpr
- $\text{makeExpr} :: (\text{a}, (\text{Expr}, (\text{Char}, (\text{Expr}, \text{b})))) \rightarrow \text{Expr}$   
 $\text{makeExpr } \_ (e1, (\text{bop}, (e2, \_))) =$   
    Op (charToOp bop) e1 e2

# Překladač výrazů (pokr.)

- $\text{charToOp} :: \text{Char} \rightarrow \text{Ops}$   
     $\text{charToOp} \text{ '+' } = \text{Add}$   
     $\text{charToOp} \text{ '-' } = \text{Sub}$   
     $\text{charToOp} \text{ '*' } = \text{Mul}$   
     $\text{charToOp} \text{ '/' } = \text{Div}$   
     $\text{charToOp} \text{ '%' } = \text{Mod}$
- $\text{isOp} :: \text{Char} \rightarrow \text{Bool}$   
     $\text{isOp } x = \text{elem } x \text{ "+-*/\%"}$

# Překladač výrazů (pokr.)

- `litParse = ((optional (token '~')) >*>  
                  (neList (spot isDigit))) `build`  
                  (charListToExpr . uncurry (++))`
- `string2num :: [Char] -> Int`  
`string2num l = doit 0 l`  
    `where doit x [] = x`  
        `doit x (c:cs) = doit (x*10 + toNum c) cs`  
        `toNum c = ord c - ord '0'`
- `charListToExpr :: [Char] -> Expr`  
`charListToExpr ('~':cs) = Lit (- string2num cs)`  
`charListToExpr cs = Lit (string2num cs)`

# Překladač výrazů (pokr.)

- `asgParser :: Parse Char Command`  
`asgParser =`  
    `(varParse >*> token '=' >*> parser) `build``  
        `(\ (var,_,ex) -> Assign (unVar var) ex)`  
    where  
        `unVar (Var v) = v`

# Překladač výrazů (pokr.)

- `commLine :: String -> Command`  
`commLine str =`  
    if null pars then Null  
    else if isOKparse pars  
            then Eval \$ fst \$ head pars  
    else if isOKparse asgp then fst \$ head asgp  
    else error "Error in input,"  
    where  
        pars = parser str  
        isOKparse [(\_,l)] = length l == 0  
        isOKparse \_ = False  
        asgp = asgParser str



# Důležité pojmy

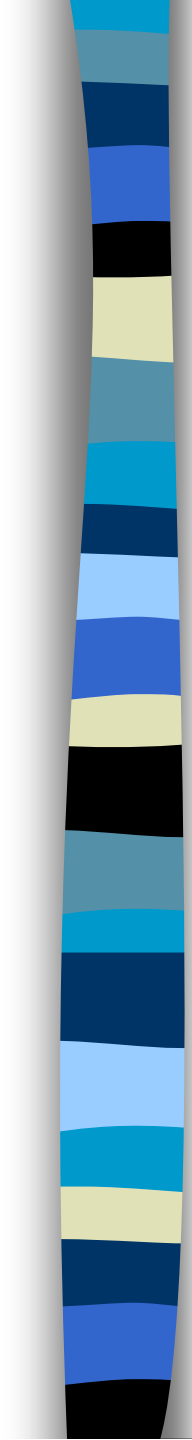
- Reprezentace dat
- Modelování
- Skládání elementárních operací
  - Základní překladač



# Úkoly



- Rozšiřte váš DB systém o základní operace jazyka SQL, zejména příkaz SELECT
- Prostudujte HSQL – <http://htoolkit.sourceforge.net>
- Vyberte si jednoduchou stolní hru (á la Člověče nezlob se) a implementujte ji v jazyce Haskell

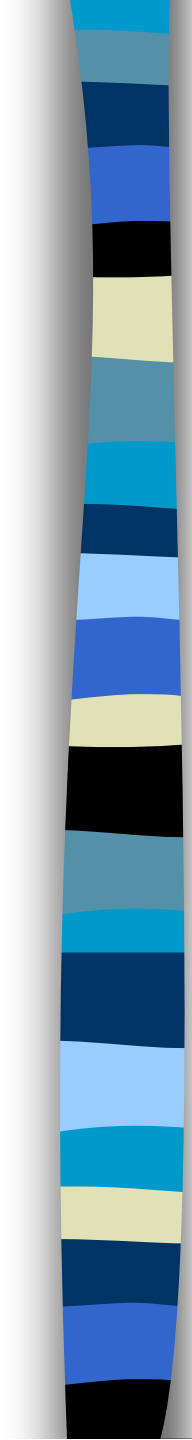


# Dokazování ve FP

# Cíle oddílu

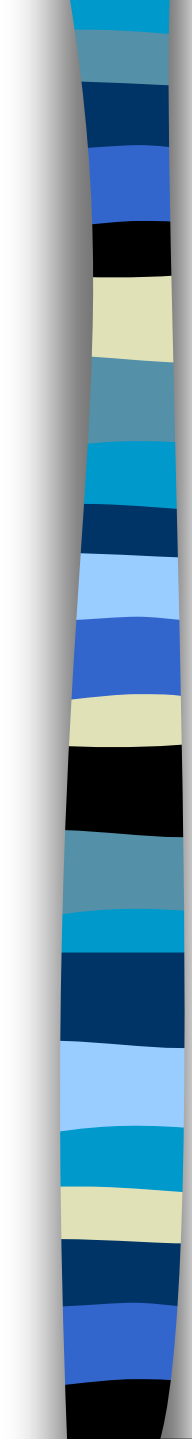


- Formální důkaz vlastnosti programu v jazyce Haskell
- Analýza efektivity programu
- Využívání „triků“



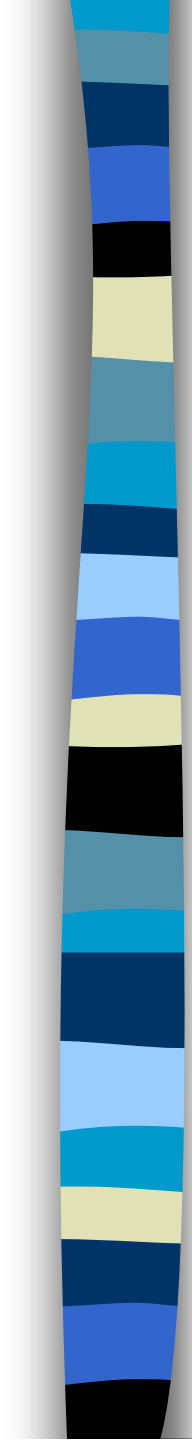
# Základní principy

- Základem je vlastnost **referenční transparency**
  - „equals can be replaced by equals“
- Princip **strukturální indukce**



# Strukturální indukce nad seznamy

- Nejprve dokážeme, že tvrzení platí pro prázdný seznam []
- Za předpokladu, že tvrzení platí pro seznam  $xs$  (indukční předpoklad) dokážeme, že platí také pro seznam  $(x:xs)$



# Praktické rady

- Krok důkazu je **vždy** tvořen aplikací některé definiční rovnice a to **v libovolném směru**
- Důležitá je volba indukční proměnné
- Závorky hrají jednu z nejvýznamnějších rolí

# Příklad

## □ Mějme dáno:

–  $(++) :: [a] \rightarrow [a] \rightarrow [a]$

$$\square ++ \text{ ys} = \text{ys} \quad (1)$$

$$(\text{x:xs}) ++ \text{ys} = \text{x:}(\text{xs} ++ \text{ys}) \quad (2)$$

## □ Asociativita:

– Pro všechny konečné seznamy  $\text{xs}$ ,  $\text{ys}$  a  $\text{zs}$  platí:

$$\bullet \text{xs} ++ (\text{ys} ++ \text{zs}) = (\text{xs} ++ \text{ys}) ++ \text{zs}$$



# Důkaz

- Použijeme strukturální indukci přes proměnnou  $xs$ , neboť definice operátoru  $++$  rozvíjí pouze první argument

- $xs = []$

$[] ++ (ys ++ zs) =$  // (1) zleva doprava

$ys ++ zs$

$([] ++ ys) ++ zs =$  // (1) zprava doleva

$ys ++ zs$

# Důkaz (pokr.)

- **$xs = (a:as)$**

$as++(ys++zs) = (as++ys)++zs$     *// předpoklad*

$(a:as)++(ys++zs) = ((a:as)++ys)++zs$

*// dokázat*

$(a:as)++(ys++zs) =$

*// (2) zleva doprava*

$a:(as++(ys++zs)) =$

*// indukční předpoklad*

$a:((as++ys)++zs) =$

*// 2 zprava doleva*

$(a:(as++ys))++zs =$

*// 2 zprava doleva*

$((a:as)++ys)++zs$

**Q.E.D.**

# Další příklady

- $[] ++ xs = xs ++ []$
- $\text{length } (xs ++ ys) = \text{length } xs + \text{length } ys$
- $\text{take } n \text{ } xs ++ \text{drop } n \text{ } xs = xs$
- $\text{rev } xs = \text{reverse } xs$ 
  - $\text{rev } [] = []$   
 $\text{rev } (x:xs) = \text{rev } xs ++ [x]$
  - $\text{reverse } xs = \text{rev}' \text{ } xs \text{ } []$   
where  $\text{rev}' \text{ } [] \text{ } ys = ys$   
 $\text{rev}' \text{ } (x:xs) \text{ } ys = \text{rev}' \text{ } xs \text{ } (x:ys)$

# Zobecnění cíle důkazu

- Mějme danou funkci:

- $\text{rev} :: [a] \rightarrow [a]$

- $\text{rev } xs = \text{shunt } xs []$

**1**

- where  $\text{shunt } [] \text{ } ys = ys$

**2**

- $\text{shunt } (x:xs) \text{ } ys = \text{shunt } xs (x:ys)$

**3**

- Dokažte, že:

- $\text{rev } (\text{rev } xs) = xs$

# První přiblížení

□ `rev (rev (x:xs))`

`= shunt (shunt (x:xs) []) []` // 1

`= shunt (shunt xs [x]) []` // 3

- **Není to vhodný cíl**

□ Efekt funkce *shunt*

- `shunt xs ys = (reverse xs) ++ ys`
- dalším otočením: `(reverse ys) ++ xs`

– **Tudíž:** `shunt (shunt xs ys) [] = shunt ys xs`

# Druhý pokus

□  $XS = []$

- $\text{shunt}(\text{shunt } [] \text{ } ys) [] = \text{shunt } ys []$  // 2

□ Indukční krok:

- $\text{shunt}(\text{shunt}(x:xs) \text{ } ys) [] = \text{shunt}(\text{shunt } xs \text{ } (x:ys)) []$  // 3

~ což by mělo dát  $\text{shunt}(x:ys) \text{ } xs$

# Druhý pokus (pokr.)

- Potom ale hypotéza musí poskytnout výsledek:

- $\text{shunt}(\text{shunt } xs \ (x:ys)) \ [] = \text{shunt } (x:ys) \ xs$

- spíše než:

- $\text{shunt}(\text{shunt } xs \ ys) \ [] = \text{shunt } ys \ xs$

- proto...

# Zesílení indukční hypotézy

- Pro všechny konečné seznamy **zs**:
  - $\text{shunt}(\text{shunt } xs \text{ } zs) [] = \text{shunt } zs \text{ } xs$
- Dostáváme se tak do místa, kdy **zs** nahrazuje **(x:ys)**



# Úspěšný pokus

□ Co nyní chceme dokázat?

– Pro všechna  $zs$ :

- $\text{shunt}(\text{shunt } xs \text{ } zs) \sqcup = \text{shunt } zs \text{ } xs$

– pro všechny konečné seznamy  $xs$  pomocí indukce

# S čím začneme

## □ První krok

- $\forall zs \text{ (shunt (shunt } [] \text{ } zs) [] = \text{shunt } zs \text{ []})}$

## □ Indukční krok

- $\forall zs \text{ (shunt (shunt (x:xs) zs) [] = \text{shunt } zs \text{ (x:xs))}$

## □ Indukční hypotéza

- $\forall zs \text{ (shunt (shunt xs zs) [] = \text{shunt } zs \text{ xs})}$

# První krok

- $\forall zs \text{ (shunt (shunt } [] \text{ } zs) [] = \text{shunt } zs \text{ []})}$

□ Důkaz:

- $\text{shunt (shunt } [] \text{ } zs) [] = \text{shunt } zs \text{ []}$  // 2

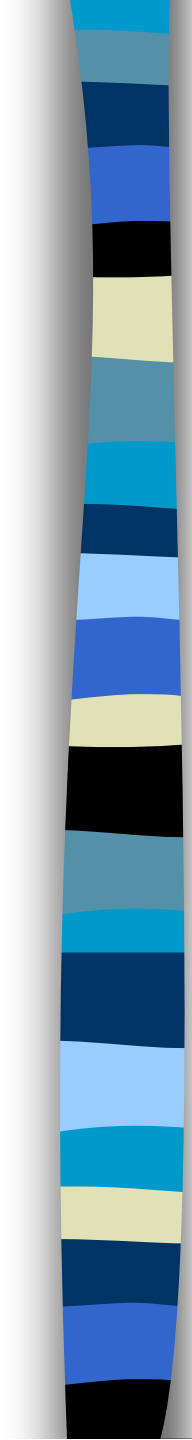
# Indukce

- $\forall zs \text{ (shunt (shunt (x:xs) zs) [] = shunt zs (x:xs))}$

## □ Důkaz:

- $\text{shunt (shunt (x:xs) zs) []} =$  // 3  
 $\text{shunt (shunt xs (x:zs)) []} =$  // hyp  
 $\text{shunt (x:zs) xs} =$  // 3  
 $\text{shunt zs (x:xs)}$

***Q.E.D.***



# Efektivita programů

## □ Asymptotická analýza

–  $T_f(x)$       počet redukcí potřebných  
pro výpočet hodnoty ( $f\ x$ )

– O-notation      „je řádu nejvýše...“

# Sémantika zápisu

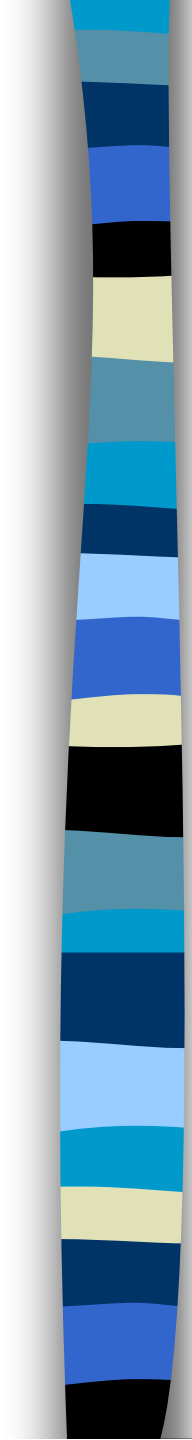
$$\begin{array}{lcl} \square & T_{\text{reverse}}(xs) & = O(n^2) \\ & T_{\text{rev}}(xs) & = O(n) \end{array}$$

□ Ze zápisu nijak nevyplývá, že *rev* je vždy rychlejší než *reverse*!

$$\begin{array}{l} \square \quad g(n) = O(h(n)) \\ \quad \equiv \exists M: \forall n > 0: |g(n)| \leq M \cdot |h(n)| \end{array}$$

# Co je $g(n)$

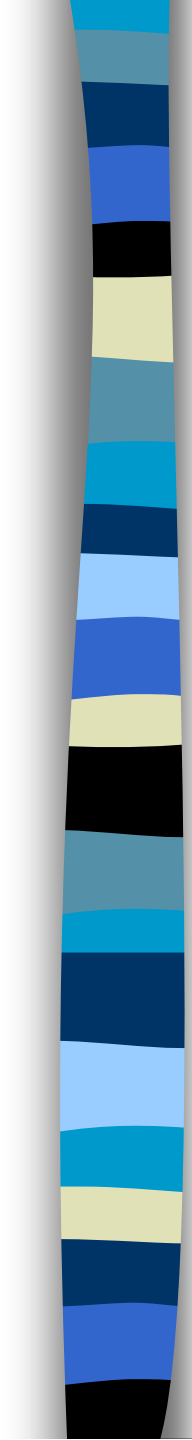
- $g(n) =$   
 $a_0 + a_1n + \dots + a_m n$   
 $\sim \rightarrow g(n) = O(n_m)$
- M z výrazu:  
 $M \cdot |h(n)|$
- je například:  
 $|a_0| + |a_1| + \dots + |a_m|$



# Význam asymptotické analýzy

- Dává výsledky nezávislé na implementaci
  - program nižšího řádu **obvykle** *jede* rychleji
- Někdy je nutný podrobnější rozbor (například u stejného řádu)
  - velikosti konstant úměrnosti, přesný počet redukcí
  - testování se stopkami





# Úkol

- Určete řády funkcí
  - hd
  - last
  - length

# Věty o dualitě (1)

- Tvoří-li + a a monoid a je-li xs konečný seznam, pak platí:
  - $\text{foldr } (\underline{+}) \underline{a} \text{ xs} = \text{foldl } (\underline{+}) \underline{a} \text{ xs}$
  - foldl a foldr definují nad monoidy stejnou funkci
  - mohou se ale lišit **efektivitou**

## Věty o dualitě (2)

- Necht'  $\underline{+}$  a  $\underline{x}$  a  $\underline{a}$  jsou takové, že pro každé  $x$ ,  $y$  a  $z$  platí:

$$x \underline{+} (y \underline{x} z) = (x \underline{+} y) \underline{x} z$$

$$x \underline{+} \underline{a} = \underline{a} \underline{x} x$$

Potom pro každý konečný seznam  $xs$  platí:

$$\text{foldr } (\underline{+}) \underline{a} \text{ } xs = \text{foldl } (\underline{x}) \underline{a} \text{ } xs$$

# Resumé

- Věta o dualitě (1) je jen speciálním případem věty o dualitě (2), kdy platí:

$$\underline{+} = \underline{X}$$

# Zamyslete se/vyzkoušejte

- $\text{sum}' = \text{foldl } (+) 0$   
 $\text{sum}'' = \text{foldr } (+) 0$
- $\text{reverse}' = \text{foldl } (\backslash l \ e \rightarrow e:l) []$   
 $\text{reverse}'' = \text{foldr } (\backslash l \ e \rightarrow l++[e]) []$
- Naznačte i prostorovou složitost
- Vysvětlete!

# Důležité pojmy

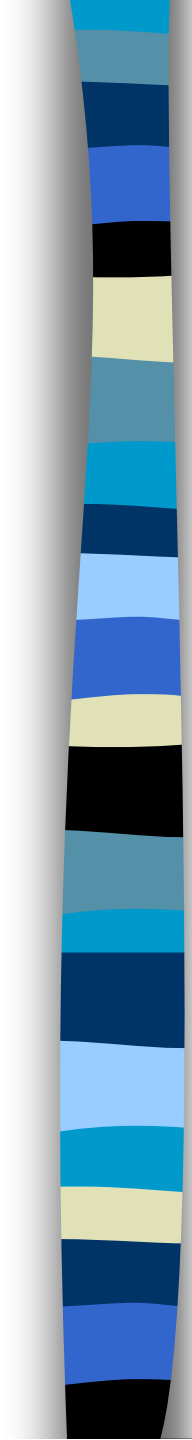
- Referenční transparence
- Strukturální indukce
  - Indukční proměnná
- Asymptotická analýza



# Úkoly



- Studujte problematiku paměťové/prostorové složitosti
- Určete časovou a paměťovou složitost operací ve vaší implementaci DB systému
- Z implementovaných úkolů vyberte 2 – 3 rekurzivní funkce pracující se seznamy a dokažte, že pracují správně



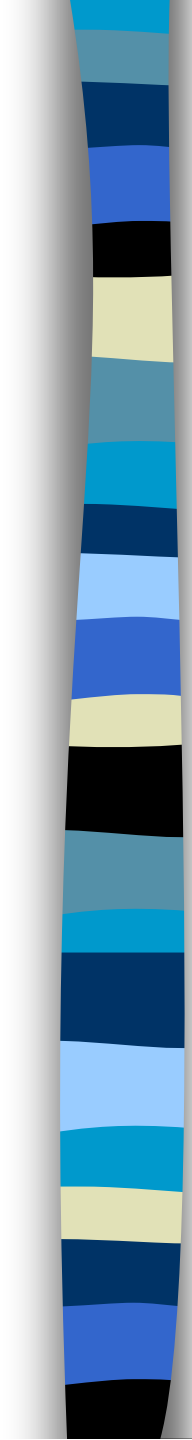
# Parallelismus v jazyce Haskell



# Cíle oddílu

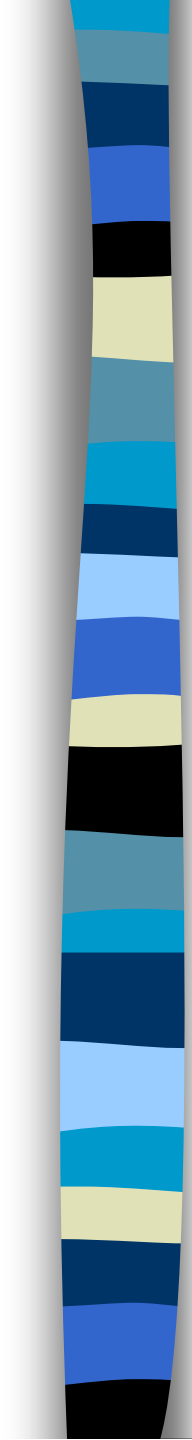


- Mechanismy paralelismu v Haskellu
  - Spuštění paralelního vlákna
  - Synchronizace vláken
  - Převzetí výsledků výpočtu



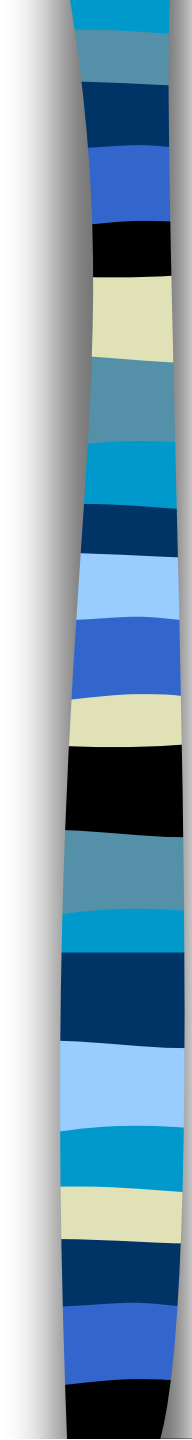
# Základní vlastnosti

- Vlákňový paralelismus
  - Procesový je dán vlastnostmi OS
  - Podpora OS pro jisté operace nutná
    - Vazba na funkce v jiném jazyce
- Vazba na monadickou třídu IO
  - *Možnost využití i třídy STM*
  - `import Control.Concurrent`
- Využití modifikovatelných paměťových míst



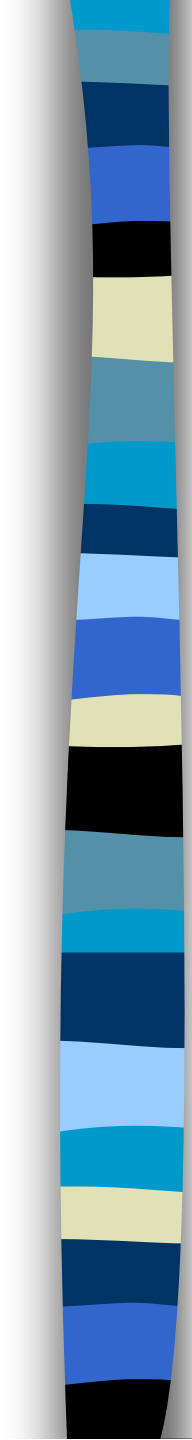
# Oddělení vlákna

- `forkIO :: IO () -> IO ThreadId`
  - Parametrem je operace, jejíž výsledkem je IO operace nad jednotkou
  - Výsledkem je identifikátor vlákna, které se od hlavního oddělilo



# Problém

- Jak zjistím, že vlákno doběhlo
- Jak získám výsledek z daného vlákna
- Jak uvolním ukončená, či běžící vlákna
- ...



# Uvolnění vlákna

- I běžící vlákno (tvrdé uvolnění)
- `killThread :: ThreadId -> IO ()`
  - Identifikátor vlákna
  - IO operace s výsledkem datové jednotky

# Získání výsledku

- Modul: `Control.Concurrent.MVar`
- `newEmptyMVar :: IO (MVar a)`
  - Vrací IO operaci obsahující prázdnou proměnnou, čerstvou, prázdnou
- `newMVar :: a -> IO (MVar a)`
  - *Vytvoří proměnnou obsahující již dodanou hodnotu*

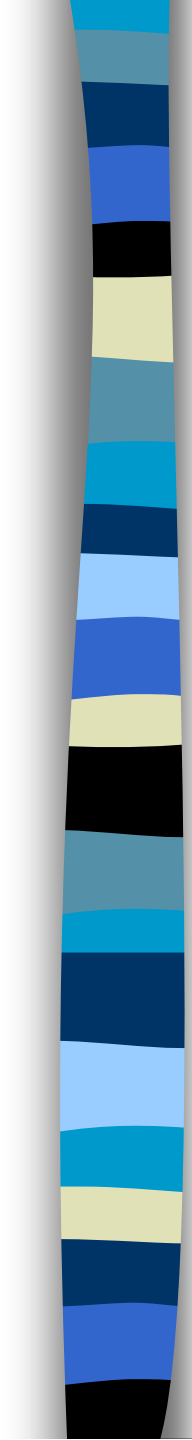
# Získání výsledku (pokr.)

- `takeMVar :: MVar a -> IO a`
  - Proměnná s hodnotou
  - IO operace obsahující výslednou hodnotu
  - *Je-li proměnná MVar prázdná, vlákno čeká na jeho naplnění*
  - *Je-li čekajících vláken více, je po naplnění vybráno jedno z nich náhodně*

# Neblokovaný přístup

- `tryTakeMVar :: MVar a -> IO (Maybe a)`
  - Proměnná s očekávanou hodnotou
  - IO akce vracející možný výsledek
    - Nothing
    - Just a





# Vytvoření vlákna – funkce vlákna

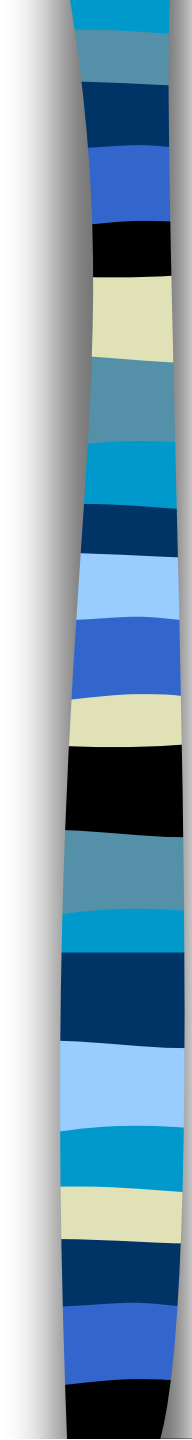
- Jeden z parametrů funkce, která tvoří vlákno je MVar
- Funkce vlákna vrátí IO ()
- Funkce vlákna nevolá cizí funkce
  - Viz konec kapitoly
- Poslední operací vlákna je zápis výsledku do předané MVar
  - Chci-li pak vlákno zrušit

# Zápis výsledné hodnoty vlákna

- `putMVar :: MVar a -> a -> IO ()`
  - Proměnná `MVar`, musí být prázdná, jinak vlákno čeká
  - Hodnota, na kterou se má nastavit
  - IO akce s výsledkem datová jednotka

# Zápis se selháním

- Více vláken se zdrojem dat
- Všechny výsledky platné
- Jedno jediné sběrné místo
- `tryPutMVar :: MVar a -> a -> IO Bool`
  - Proměnná
  - Hodnota, na kterou se má nastavit
  - IO akce – pravda znamená úspěšný zápis



# Další operace s proměnnými MVar

- Čtení
  - Modifikace
  - Test na prázdnotu
  - ...
- 
- *Viz dokumentaci jazyka Haskell*

# Příklad

## □ Balastní funkce

- Simulace zátěže výpočtu
- Lze dosadit libovolnou jinou

```
□ sumAllSums [] = 0
 sumAllSums l@(_:xs) = sumlist 0 l + sumAllSums xs
 where sumlist res [] = res
 sumlist sr (v:vs) = sumlist (sr+v) vs
```

# Příklad (pokr.)

## □ Hlavní funkce programu – začátek:

```
□ main = do
 putStrLn "Starting..."
 mv1 <- newEmptyMVar
 mv2 <- newEmptyMVar
```

Jedná se  
monadickou  
operaci

Dvě výpočetní  
vlákna, dva  
výsledky

# Příklad (pokr.)

- Hlavní funkce programu – spuštění:

```
□ main = do
```

```
...
```

```
t1 <- forkIO $ mkSum1 mv1
```

```
t2 <- forkIO $ mkSum2 mv2
```

Oddělení vlákn

Vazba na  
proměnné jako  
parametr

# Příklad (pokr.)

- Hlavní funkce programu – sběr a tisk výsledků:

- `main = do`

```
...
s2 <- takeMVar mv2
s1 <- takeMVar mv1
putStrLn $ "Suma1: " ++ show s1
putStrLn $ "Suma2: " ++ show s2
```

Dva výsledky

Tisk  
výsledných  
hodnot  
(zpracování)



# Příklad (pokr.)

- Hlavní funkce programu – ukončení programu:

```
□ main = do
 ...
 forkIO $ do killThread t1
 killThread t2
 putStrLn "Done!"
```

Dvě vlákna

Poslední  
operace  
programu

# Příklad (konec)

- Hlavní funkce programu – lokální funkce:

```
□ main =
 ...
 where
```

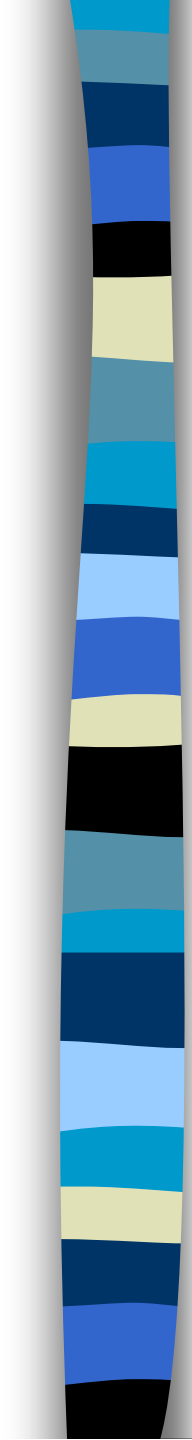
Uložení  
výsledku

Balastní funkce

Různé  
parametry pro  
porovnání

```
mkSum1 mv = do
 let res = sumAllSums [1..10000]
 res `seq` putMVar mv res

mkSum2 mv = do
 let res = sumAllSums [1..10001]
 res `seq` putMVar mv res
```



# Odložení vlákna

- `yield :: IO ()`
  - IO akce poskytující datovou jednotku
  - Vynucuje změnu kontextu
  - Kooperativní vícevláknové zpracování

# Příklad

## □ Tělo funkce main (drobná úprava)

```
□ main = do
 putStrLn "Starting..."
 mv1 <- newEmptyMVar
 mv2 <- newEmptyMVar
 t1 <- forkIO $ mkSum1 mv1
 t2 <- forkIO $ mkSum2 mv2
 yield
 takeRes mv1 mv2
 forkIO $ do killThread t1
 killThread t2
 putStrLn "Done!"
```

V hlavním  
vlákně se  
neodehrává  
výpočet

# Příklad (konec)

## □ Funkce takeRes (vyzvednutí výsledku):

```
□ takeRes mv1 mv2 = do
 mr1 <- tryTakeMVar mv1
 case mr1 of
 Just r1 -> takeMVar mv2
 Nothing -> do
 yield
 mr2 <- tryTakeMVar mv2
 case mr2 of
 Just r2 -> takeMVar mv1
 Nothing -> do
 yield
 takeRes mv1 mv2
```

Stále se asi počítá, ale pokud bych potřeboval, mohu i já, nejsem blokován

# Využití u datových struktur

- `data Tree a = Node a (Tree a) (Tree a)`  
                  | `Leaf`
- Úkol: spočtete hloubku stromu
- Sekvenčně:
  - `sdepth Leaf = 0`  
   `sdepth (Node _ l r) = if ld>rd then ld else rd`  
   where  
      `ld = 1 + sdepth l`  
      `rd = 1 + rdepth r`

# Paralelně

## ❑ Špatný přístup:

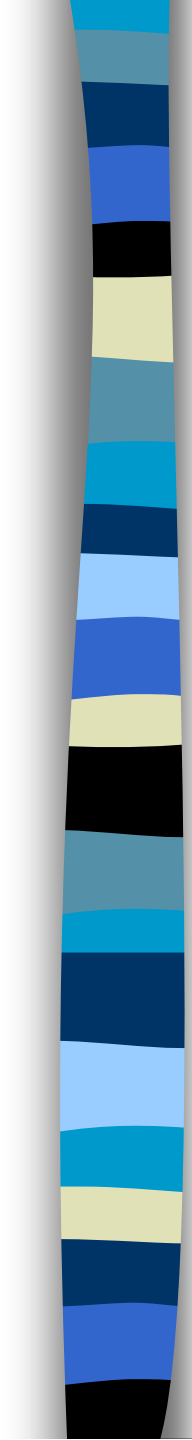
```
❑ pdepth Leaf = return 0
 pdepth (Node _ l r) = do
 mv1 <- newEmptyMVar
 mv2 <- newEmptyMVar
 t1 <- forkIO $ pdepth l >>= putMVar mv1
 t2 <- forkIO $ pdepth r >>= putMVar mv2
 ld <- takeMVar mv1
 rd <- takeMVar mv2
 killThread t1
 killThread t2
 return $ 1 + if ld>rd then ld else rd
```

# Paralelně

## □ Korektní přístup:

```
□ pdepth' Leaf = return 0
 pdepth' (Node _ l r) = do
 mv1 <- newEmptyMVar
 mv2 <- newEmptyMVar
 t1 <- forkIO $ putMVar mv1 $! sdepth l
 t2 <- forkIO $ putMVar mv2 $! sdepth r
 ld <- takeMVar mv1
 rd <- takeMVar mv2
 killThread t1
 killThread t2
 return $ 1 + if ld>rd then ld else rd
```





# Zhodnocení

- Špatný přístup
  - Příliš mnoho vláken
  - Využití cílového HW malý
- Korektní přístup
  - Právě 2 výpočetní vlákna
    - Třetí čeká na hodnotu, šlo by vylepšit
  - Vyšší využití HW pro vlastní výpočet

# Vazba na OS

- Pro vazbu na vlákna OS
  - Volání cizích funkcí
  - Vlastnosti OS
  - ...
- `forkOS :: IO () -> IO ThreadId`
  - Funkce vlákna vracející IO akci s výsledkem datové jednotky
  - IO akce s výsledkem identifikátoru vlákna

# Důležité pojmy

## □ Vlákno

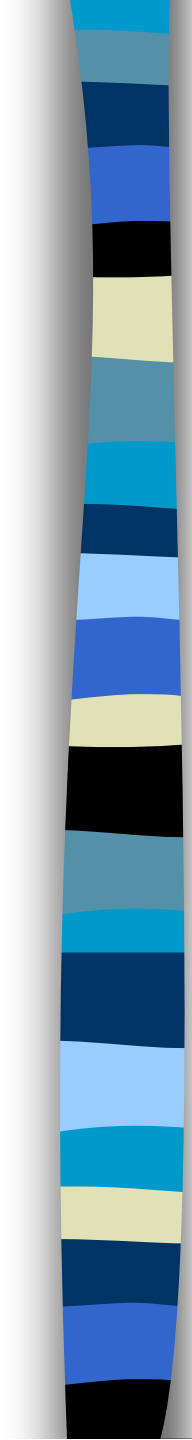
- Oddělení
- Synchronizace
- Předání výsledku
- Změna kontextu



# Úkoly



- Prostudujte knihovnu Concurrent
- Které z dosud implementovaných funkcí se nabízejí pro paralelizaci (2 výpočetní vlákna)?
- Implementujte paralelní verze pro 2 vámi dosud vytvořené funkce a ověřte jejich správnou činnost i zrychlení výpočtu.

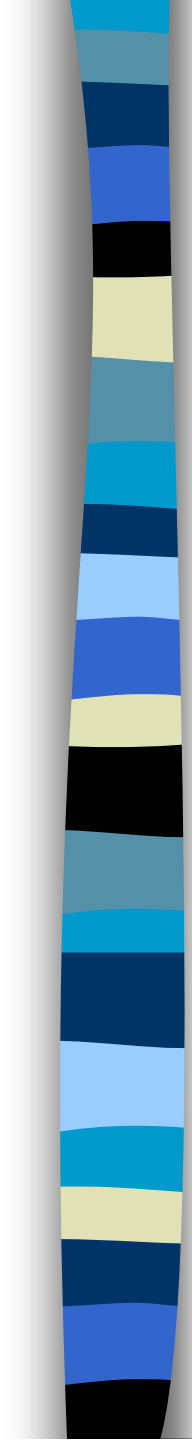


# Denotační sémantika

# Cíle oddílu

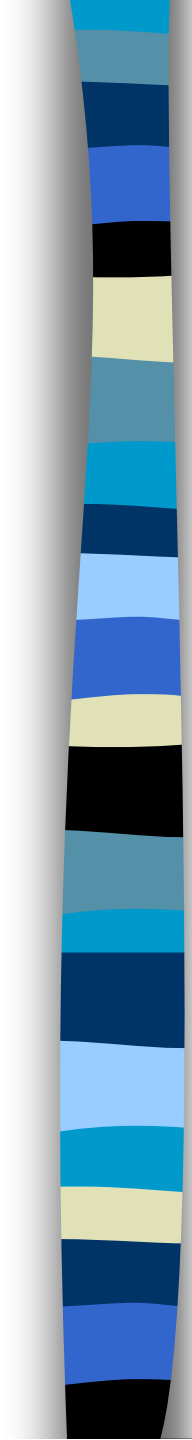


- Porozumění denotační sémantice
  - Čtení zápisu
  - Tvorba velmi jednoduchých popisů



# Co je denotační sémantika

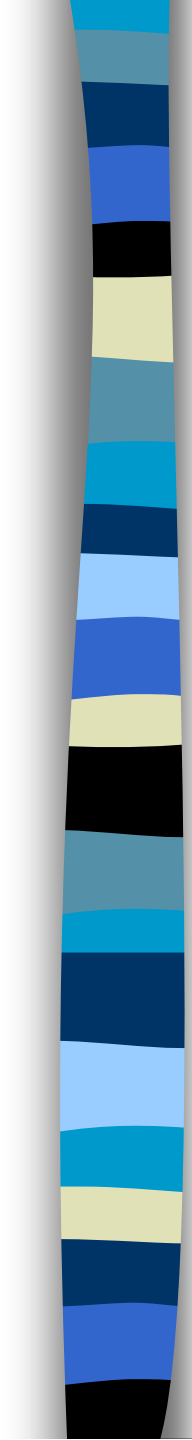
- Denotační sémantika programovacího jazyka popisuje význam programu jako *funkci*
  - program: *Input* → *Output*



# Princip kompozicionality

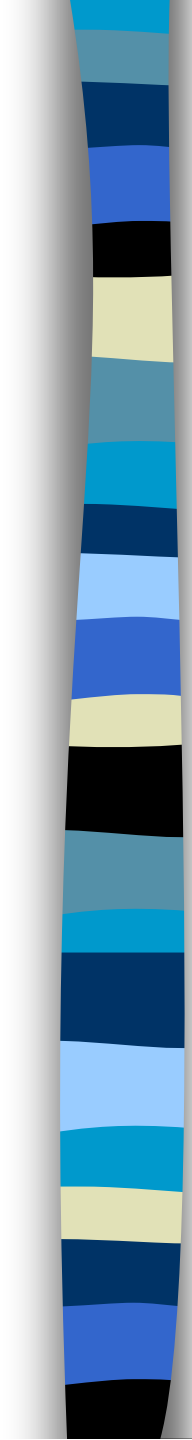
- Význam složené konstrukce je dán kombinací významů jednotlivých složek





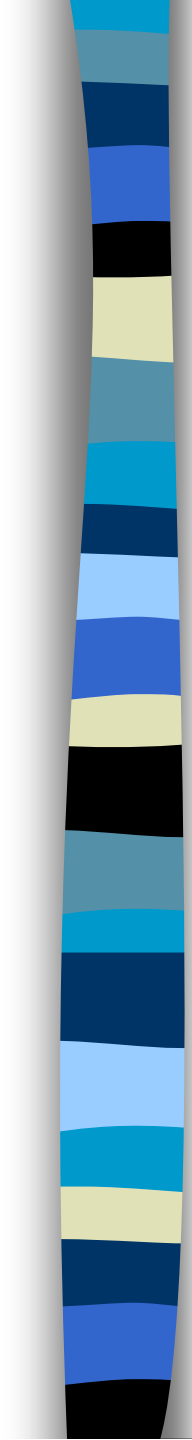
# Popisné prostředky

- Pro popis se používá  $\lambda$ -kalkul
- Lze jednoduše vyjádřit také pomocí *funkcionálního jazyka*



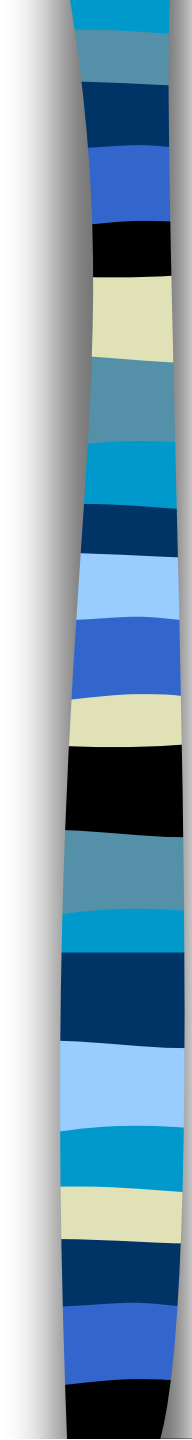
# Abstraktní syntax

- Popisuje zjednodušenou strukturu programu bez detailů, které nenesou žádnou sémantickou informaci
  - priorita a asociativita operátorů
  - závorky
  - ...



# Abstraktní syntax (pokr.)

- Pro datovou realizaci abstraktní syntaxe programu se používá obvykle stromová struktura - **abstraktní syntaktický strom (AST)**



# Domény

- Syntaktické konstrukce jsou strukturovány do **syntaktických domén**
  - Doména:
    - výrazů - **Exp**
    - příkazů - **Com**
    - deklarací - **Dec**
    - programů - **Prog**
    - ...

# Sémantické funkce

- Přiřazují význam jednotlivým syntaktickým konstrukcím
- Jsou definovány pro každou doménu zvlášť:
  - $e[1 + 1] = 2$
  - $p[i = \text{read}; \text{write } 2 \times i] = \lambda (x : \_).[2 \times x]$

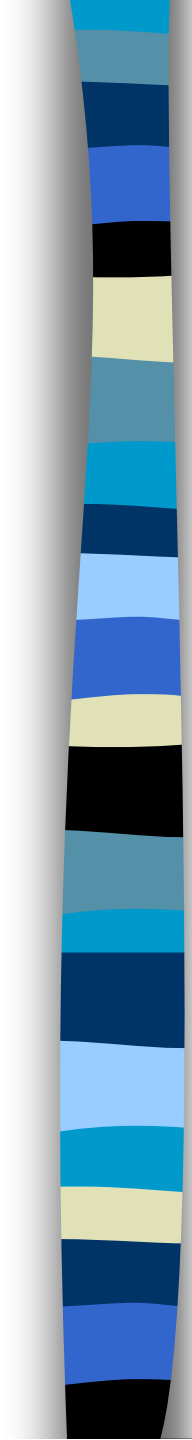
# Výrazy s konstantami

- Významem výrazu je jeho hodnota:

$$- e[1+1] = 2$$

# Reprezentace ve FJ

- `data Exp = Add Exp Exp`  
                  | `Mul Exp Exp`  
                  | `Neg Exp`  
                  | `Num Int`
- `e :: Exp -> Int`  
  `e (Add e1 e2) = (e e1) + (e e2)`  
  `e (Mul e1 e2) = (e e1) * (e e2)`  
  `e (Neg e1)     = 0 - (e e1)`  
  `e (Num x)      = x`



# Výrazy s proměnnými

- Význam výrazu (jeho hodnota) závisí na konkrétních hodnotách proměnných
- Funkce přiřazující proměnným hodnotu (valuační funkce) musí být parametrem sémantické funkce  $e$  - tato funkce modeluje paměť počítače s pojmenovanými buňkami

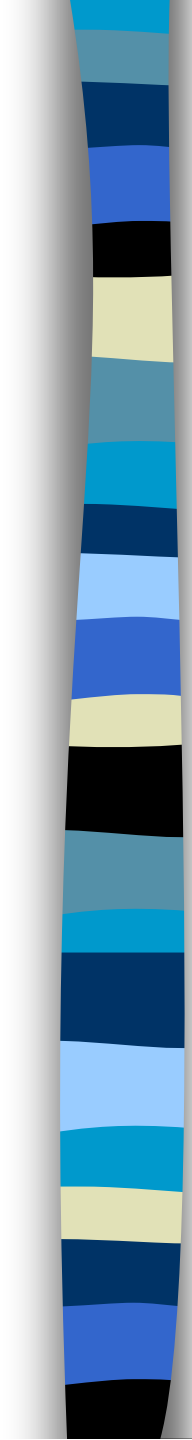


# Reprezentace ve FJ

- `type Store = String -> Int`  
  `data Exp = ...`  
      `| Var String`

- `e :: Exp -> Store -> Int`

```
e (Add e1 e2) s = (e e1 s) + (e e2 s)
e (Num x) _ = x
e (Var v) s = s v
```



# Výrazy s přiřazením

- Hodnoty proměnných (a tedy i valuační funkce) se mohou během vyhodnocení výrazu změnit
- Změna valuační funkce musí být součástí funkční hodnoty sémantické funkce  $e$  (t.j. výsledek musí být uspořádanou dvojicí hodnot)

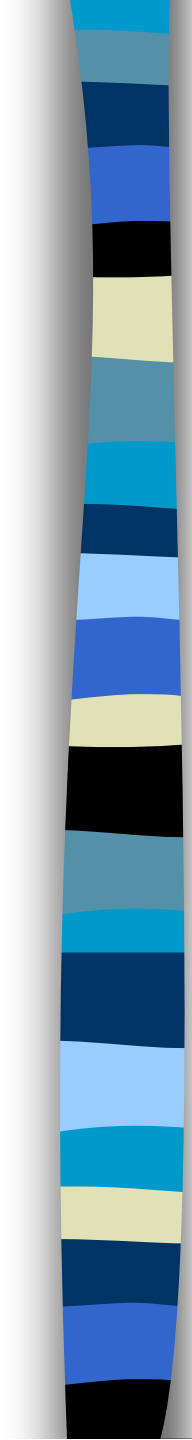
# Reprezentace ve FJ

- `data Exp = ...  
          | Asgn String Exp`
- `e :: Exp -> Store -> (Int, Store)`
- `e (Asgn v e1) s =  
    let (v1, s') = e e1 s  
        s' ' v' = if v' == v then v1  
                  else s v'  
    in (v1, s' ' )`

# Reprezentace ve FJ (pokr.)

- $e \text{ (Add } e1 \text{ } e2) \text{ } s =$   
     $\text{let } (v1, s') = e \text{ } e1 \text{ } s$   
     $\text{in let } (v2, s'') = e \text{ } e2 \text{ } s'$   
     $\text{in } (v1+v2, s'')$

$e \text{ (Num } x) \quad s = (x, s)$



# Příkazy

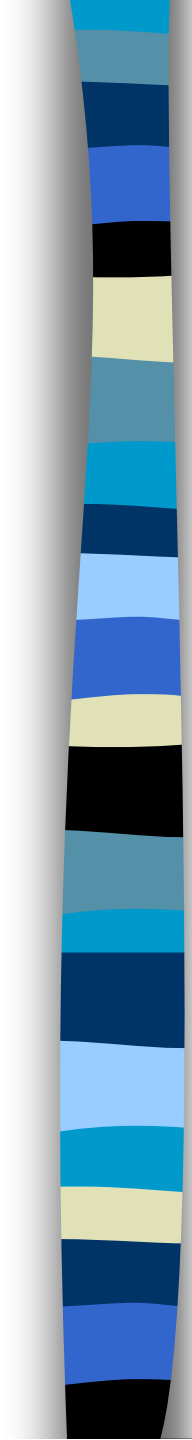
- Příkaz neprodukuje (na rozdíl od výrazu) hodnotu, jeho významem je vedlejší efekt - změna stavu programu (např. hodnot proměnných)

# Reprezentace ve FJ

- $\text{Data Com} = \text{Eval Exp}$   
          |  $\text{If Exp Com}$   
          |  $\text{While Exp Com}$   
          |  $\text{Seq Com Com}$
  - $c :: \text{Com} \rightarrow \text{Store} \rightarrow \text{Store}$
- $c \text{ (Eval } e1) \text{ } s =$   
     $\text{let } (\_, s') = e \text{ } e1 \text{ } s$   
     $\text{in } s'$

# Reprezentace ve FJ (pokr.)

- $c \text{ (If } e1 \text{ } c1) \text{ } s =$   
     $\text{let } (v1, s') = e \text{ } e1 \text{ } s$   
     $\text{in if } v1 == 0 \text{ then } s'$   
     $\text{else } c \text{ } c1 \text{ } s'$
- $c \text{ (While } e1 \text{ } c1) \text{ } s =$   
     $\text{let } (v1, s') = e \text{ } e1 \text{ } s$   
     $\text{in if } v1 == 0 \text{ then } s'$   
     $\text{else } c \text{ (While } e1 \text{ } c1) (c \text{ } c1 \text{ } s')$
- $c \text{ (Seq } c1 \text{ } c2) \text{ } s = c \text{ } c2 \text{ (c } c1 \text{ } s)$



# Vstup a výstup

- Stav programu je tvořen nejen okamžitými hodnotami proměnných, ale také nezpracovaným vstupem a již vyprodukovaným výstupem
- Místo typu **Store** je třeba v předchozích definicích používat typ **State** zahrnující i vstup a výstup programu



# Reprezentace ve FJ

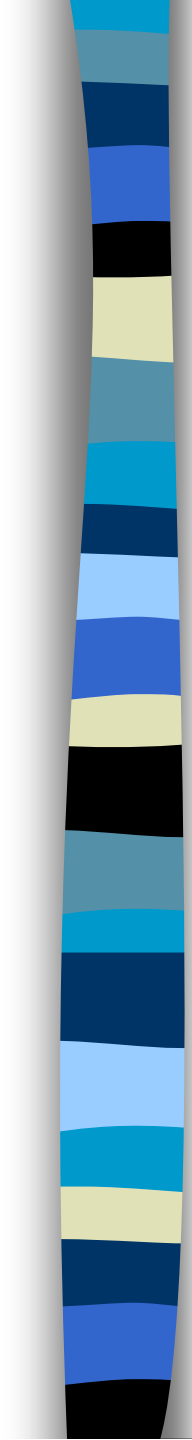
- `type Input = [Int]`  
  `type Output = [Int]`  
  `type State = (Store, Input, Output)`
- `data Exp = ...`  
    `| Read`
- `e :: Exp -> State -> (Int, State)`  
  
  `e Read (s, x:xs, o) = (x, (s, xs, o))`

# Reprezentace ve FJ (pokr.)

- `data Com = ...  
          | Write Exp`

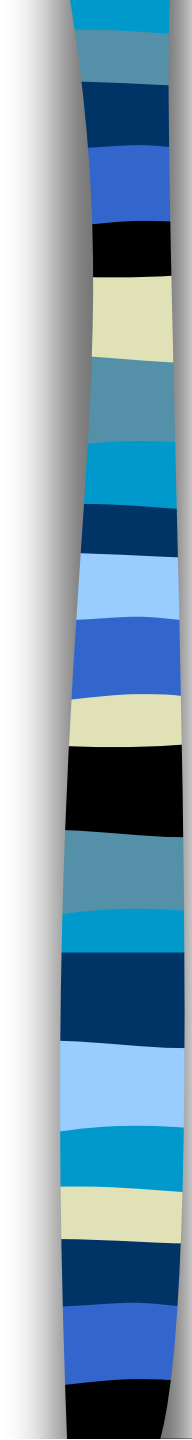
- `c :: Com -> State -> State`

```
c (Write e1) s =
 let (v1, (s', i, o)) = e e1 s
 in (s', i, v1:o)
```



# Kontinuace

- Význam „zbytku programu“ od aktuálního místa v programu až po jeho ukončení lze modelovat **kontinuací** - funkcí, která na základě aktuálního stavu vrátí výsledek celého programu



# Kontinuace v denotační sémantice

- Sémantická funkce obdrží jednu nebo více kontinuací, kterým po vyhodnocení může (a nemusí) předat řízení

# Kontinuace výrazu

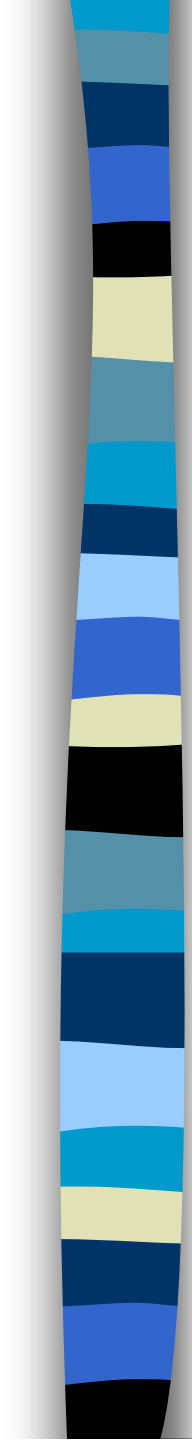
- Obdrží hodnotu výrazu, stav programu po jeho výpočtu a vrátí výsledek programu

- `type ECont = Int -> State -> Output`

# Kontinuace příkazu

- Obdrží stav po provedení příkazu a vrátí výsledek programu

- `type CCont = State -> Output`



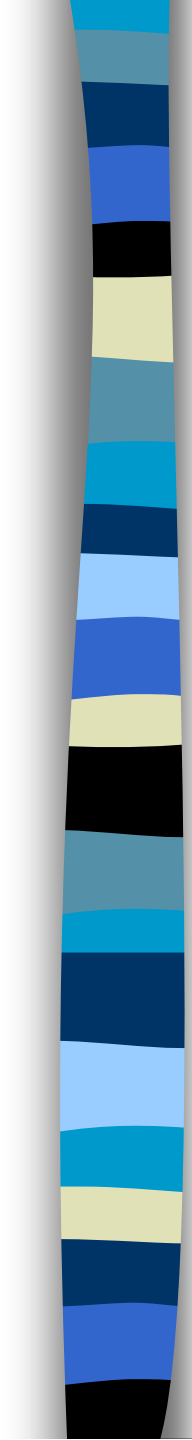
# Sémantická funkce pro výraz

- Obdrží zpracováváný výraz, stav před jeho výpočtem a kontinuuaci, které má předat nový stav a výsledek; vrací výsledek celého programu („vyrobený“ obvykle kontinuuací)

# Reprezentace ve FJ

- $e :: \text{Exp} \rightarrow \text{State} \rightarrow \text{ECont} \rightarrow \text{Output}$
- $e \text{ (Add } e1 \text{ } e2) \text{ } s \text{ } ec =$   
     $e \text{ } e1 \text{ } s \text{ } (\backslash v1 \text{ } s' \rightarrow$   
         $e \text{ } e2 \text{ } s' \text{ } (\backslash v2 \text{ } s'' \rightarrow$   
             $ec \text{ } (v1+v2) \text{ } s''))$



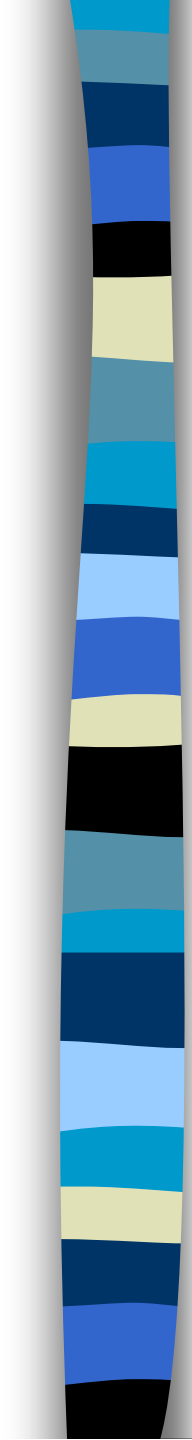


# Sémantická funkce pro příkaz

- Obdrží zpracováváný příkaz, stav před jeho provedením a kontinuaci, které má předat nový stav; vrací výsledek celého programu

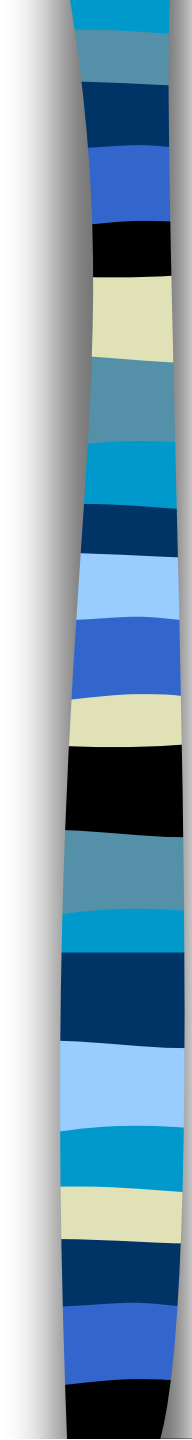
# Reprezentace ve FJ

- `c :: Com -> State -> CCont -> Output`
- `c (Seq c1 c2) s cc =  
 c c1 s (\s' ->  
 c c2 s' (\s'' -> cc s''))`
- `c (If e1 c1) s cc =  
 e e1 s (\v1 s' ->  
 if v1 == 0 then cc s'  
 else c c1 s' cc)`



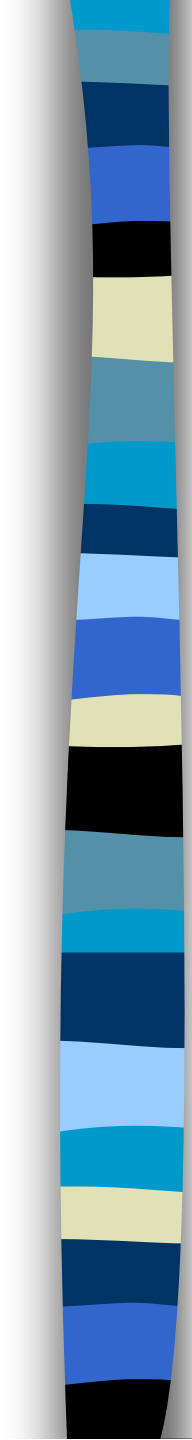
# Použití kontinuací

- *Reakce na chybové stavy* - funkce nepředá řízení žádné kontinuaci a tím ukončí vyhodnocení
- *Definice významu skokových příkazů* - při skoku se předá řízení kontinuaci odpovídající cílovému místu skoku



# Použití kontinuací (pokr.)

- *Definice významu volání podprogramu -*  
kontinuace odpovídající zbytku  
programu za příkazem volání se  
uschová pro použití v příkazu **return**

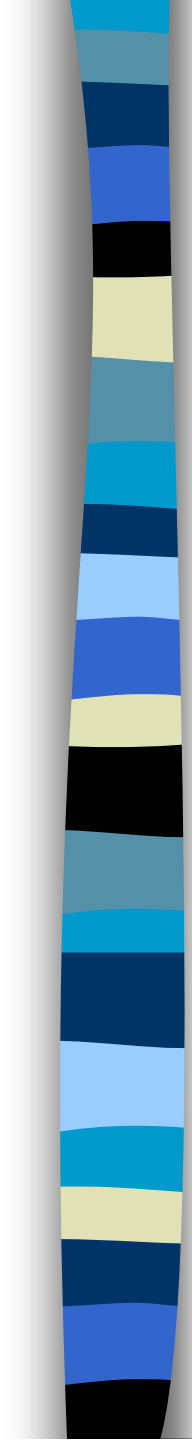


# Modely výpočtu

- Program nejprve provede výpočet a pak zobrazí výsledek nebo chybu
- Program vypisuje průběžně výsledky a na konci oznámí úspěch nebo chybu

# Syntax a sémantika programu

- Program je tvořen posloupností příkazů
  - `type Prog = Com`
- Vstupem programu je posloupnost čísel, výstupem posloupnost čísel zakončená znakem OK nebo chybovou zprávou
  - `type Input = [Int]`  
`data Value =`  
`I Int | OK | Err String`  
`type Output = [Value]`



# Syntax a sémantika programu ...

- Stav programu je tvořen obsahem proměnných a nepřečteným vstupem; na počátku není žádná proměnná definována

# Reprezentace ve FJ

- `type Store = String -> Maybe Int`  
`data State = State {store::Store,`  
`input::Input}`
- `emptyStore :: Store`  
`emptyStore id = Nothing`
- `initialState :: Input -> State`  
`initialState inp =`  
`State store=emptyStore, input=inp`



# Syntax a sémantika programu ...

## □ Vyhodnocení programu

- $p :: \text{Prog} \rightarrow \text{Input} \rightarrow \text{Output}$

```
p body inp =
 c body (initialState inp)
 (_ -> [OK])
```

# Syntax a sémantika programu ...

## □ Ošetření nedefinované proměnné

- $e :: \text{Exp} \rightarrow \text{State} \rightarrow \text{ECont} \rightarrow \text{Output}$

```
e (Var v) s ec =
 case (store s) v of
 Just v1 -> ec v1 s
 Nothing -> [Err ("Undef: "++v)]
```

# Syntax a sémantika programu ...

## □ Realizace příkazu **Write**

- $c :: \text{Exp} \rightarrow \text{State} \rightarrow \text{CCont} \rightarrow \text{Output}$

```
c (Write e1) s cc =
 e e1 s (\v1 s' -> (I v1) :
 (cc
 (State (store s'), (input s')))))
```

# Důležité pojmy

- Denotační sémantika
  - Syntaktická doména
  - Sémantická funkce
- Kontinuace
- Model výpočtu



# Úkoly



- Definujte abstraktní syntaxi a sémantiku podmíněného výrazu if-then-else
  - bez použití kontinuuací
  - s kontinuuacemi
- Definujte abstraktní syntaxi a sémantiku příkazu repeat-until
  - bez použití kontinuuací
  - s kontinuuacemi

